COMPLETE WALKTHROUGH

OptiChain

Demand forecasting and stock planning for a real retail chain. Every step, every graph, every model and every number of the project, explained in depth: what it is, how it works, and why it was chosen.

Saved vs the official method	Over a year (estimate)	Stock on shelves	Forecast error
€708	**≈ €6,100**	**−45%**	**57.1%**
in 6 weeks · 16% cheaper	if the test weeks are typical	1,487 vs 2,713 units a week	LightGBM · benchmark 63.7%

Contents

1 · The problem
2 · The data
3 · Data audit and cleaning
4 · Exploratory data analysis
5 · Time-series analysis (Box-Jenkins)
6 · The ten forecasting models
7 · Testing the forecasts (backtest)
8 · Uncertainty
9 · The inventory simulator
10 · The ordering policy
11 · The final test
12 · The stock outlook
13 · The dashboard, page by page
14 · The engineering
15 · Limitations and next steps
Appendix A · Glossary
Appendix B · Questions and answers
Appendix C · The notebook, cell by cell

Data: Vandeput, N. (2025). *VN2 Inventory Planning Challenge*. Organised by Nicolas Vandeput (SupChains); sponsored by Flieber, Daybreak and SupChains. https://supchains.com/vn2-inventory-competition/ · CC BY 4.0.

PART 1

The problem

1.1 · The business situation

A retail chain has 67 stores. Each store sells some of 297 products. Every Monday, before the stores open, the planner orders stock for every product in every store from the central warehouse. The order is delivered **two weeks later**. Customers then buy what is on the shelf. If the shelf is empty, the customer leaves and the sale is **lost** (there is no waiting list, no back-order).

1.2 · The two costs

Event	Cost	Real meaning
A unit is still on the shelf at the end of a week	€0.20 per unit per week	money tied up in stock, shelf space, risk of damage or expiry (the **holding cost**)
A customer wants a unit but the shelf is empty	€1.00 per unit	lost margin and an unhappy customer (the **shortage cost**)

A missed sale is **five times** more expensive than keeping a unit for a week. That ratio drives every decision later: it is better to keep a little extra stock than to run out, but not too much.

1.3 · Why it is hard

- **The future is unknown.** Orders are placed two weeks ahead, so the planner must predict demand.
- **Demand is small and irregular.** The typical store-product sells about one unit a week, often zero.
- **The data hides demand.** When the shelf was empty, the sales record shows zero even if customers came.
- **Stock already on its way matters.** Each order must account for stock on the shelf and orders not yet delivered, which are themselves eaten by the next two weeks of demand.
- **Many items.** 599 store-products, each needing its own decision every week.

1.4 · The goal of the project

Build a system that **(1)** learns from three years of weekly sales, **(2)** predicts the coming weeks for every store-product, **(3)** turns predictions into orders that minimise the total of the two costs, **(4)** proves it on real weeks it never saw, scored with the official competition rules, and **(5)** presents everything in an explanatory notebook and an interactive dashboard.

> **In one sentence:** predict how much each product will sell, then order just enough, not too little (empty shelves cost €1 a unit) and not too much (leftovers cost €0.20 a unit per week).

Data: VN2 Inventory Planning Challenge (Vandeput, 2025), CC BY 4.0

PART 2

The data

2.1 · Source and choice

The data is the **VN2 Inventory Planning Challenge**, a real competition organised in 2025 by Nicolas Vandeput (SupChains), sponsored by Flieber and Daybreak and hosted by DataSource.ai. About 180 teams took part. The sales are real weekly sales of an anonymous retail chain. The organiser published the data, the official simulator and the official benchmark on GitHub (SupChains/VN2) under the CC BY 4.0 licence.

The project first used the Walmart M5 dataset (2011-2016). It was replaced because M5 has no stock levels, no costs and no ordering rules, so those had to be invented. VN2 brings **real stock, real costs, real lead time and an official score** to compare with 180 real teams. The M5 version is kept in *archive/m5*.

2.2 · Size and time span

Item	Value
Stores / products / store-products	67 / 297 / 599
Weeks in total	165 (Mondays from 2021-04-12 to 2024-06-03)
History (used for learning)	157 weeks, up to week 0 = 8 April 2024
Competition (final test only)	8 weeks: week 1 = 15 April 2024 ... week 8 = 3 June 2024
Units sold in total	290,618

2.3 · The files

File	Content	How it is used
Week 0 - 2024-04-08 - Sales.csv	157 weeks of sales per store-product	what planners knew before round 1
Week 1 ... Week 8 - Sales.csv	the same, one more week each round	Week 8 holds every week: demand of the simulation
Week 0 - In Stock.csv	True/False per week: was stock left at the end of the week?	finds the censored weeks
Week 0 - Master.csv	ProductGroup, Division, Department, DepartmentGroup, StoreFormat, Format	inputs of the machine-learning model; charts by group
Week 0 - Initial State.csv	stock on the shelf and orders in transit at the end of week 0	the real starting point of the simulation
Benchmark order files (6)	the official benchmark's orders	verify our simulator to the cent

2.4 · The shape of the data

Each row is one **store-product** (one product in one store), identified by two anonymised integer codes, *Store* and *Product*. Each column is one week, labelled with its Monday date. A cell holds the number of units sold that week. The in-stock file has exactly the same shape with True/False values.

Store	Product	2021-04-12	2021-04-19	...	2024-06-03
0	126	0	1	...	1
0	182	0	0	...	0
...	...	...	...	...	...

2.5 · Why the stores and products are numbers

The company is anonymous. The data gives codes, not names. The project never invents names: on the dashboard, five example products get descriptive nicknames based on how they behave (for example "The Christmas Rocket", which sells 2.8 times more in December), always shown next to their real code.

2.6 · The competition game

Round	Planner knows sales up to	Order placed before	Order arrives
1	week 0 (8 Apr)	week 1 (15 Apr)	week 3 (29 Apr)
2	week 1	week 2	week 4
3	week 2	week 3	week 5
4	week 3	week 4	week 6
5	week 4	week 5	week 7
6	week 5	week 6	week 8

Weeks 1 and 2 are served only by the stock and orders that already exist, so their cost (€913.80) is the same for every player. The **score is the total cost of weeks 3 to 8**; lower is better.

PART 3

Data audit and cleaning

3.1 · Data-quality audit

Fact	Value
Store-product rows	599
Stores	67
Products	297
Weeks in total	165
History weeks (to week 0)	157
Competition weeks (1-8)	8
First week	2021-04-12
Last week	2024-06-03
Missing values	0
Negative sales	0
Non-integer sales	0
Units sold in total	290618
Weeks with zero sales	43.2%
Weeks ending out of stock	10.6%
Weeks before launch	7.2%
Rows ever out of stock	80.6%

The data is technically perfect: no missing cells, no negative or fractional sales, and the nine weekly files agree with each other (each repeats the earlier weeks unchanged and adds one week). But two facts stand out: **43.2%** of weeks sell nothing, and **10.6%** of weeks are flagged out of stock (80.6% of store-products ran out at least once).

3.2 · Hidden missing value 1: weeks before launch

7.2% of all cells are zeros before a product's first ever sale in that store. The product was not on the shelf yet, so those zeros say nothing about demand. 38% of store-products sell from the first week; 371 were launched later (median 4 weeks in). **Rule:** every week before the first sale is set to missing (NaN).



Figure 1. When products were launched. Each bar counts store-products whose first sale fell in that week.

3.3 · Hidden missing value 2: out-of-stock weeks (censored demand)

When a week ends with an empty shelf, the recorded sales are a **lower bound** of demand: some customers found nothing. Statisticians call this **censored demand**. In this data the effect is extreme: **100% of out-of-stock weeks recorded zero sales**; the shelf was empty the whole week. In the history, 4.3% of weeks after launch are censored. **Rule:** these weeks are set to missing (NaN). The result is called **clean demand**.

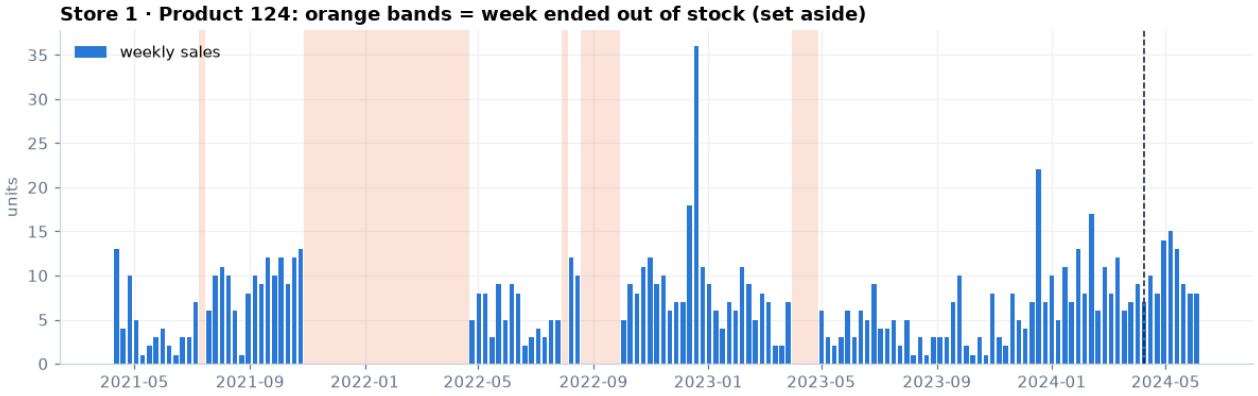


Figure 2. One store-product with frequent stockouts. Orange bands mark weeks that ended out of stock: they fall exactly on the zero-sales weeks, which is why those zeros are not treated as demand.

> **Why not fill the gaps with zero?** A zero would teach every model that demand is lower than it really is, so the plan would under-order and run out even more often. **Why not fill them with a guess?** An invented number would look like real information. The honest choice is to leave them out and let each method skip them.

Method	Treatment of a missing week
Moving averages, exponential smoothing, Croston, TSB	skipped: averages use clean weeks only
ARIMA / SARIMA	kept missing: the Kalman filter of the state-space model skips it exactly
ADF, KPSS, STL, ACF/PACF (need an unbroken series)	linear interpolation between neighbouring clean weeks
LightGBM	never used as a training target; missing lag features are handled by the trees

3.4 · Demand patterns (Syntetos-Boylan classification)

Each store-product is described by two numbers computed on its clean history:

ADI = number of clean weeks ÷ number of weeks with a sale CV^2 = (standard deviation ÷ mean of the non-zero sales)^2

ADI is the average gap between sales (1 = sells every week). CV² measures how much the size of a sale varies. With the standard cut-offs (ADI 1.32 and CV² 0.49) each store-product falls into one of four patterns:

Pattern	Rule	Plain meaning	Store-products
Smooth (steady seller)	ADI < 1.32, CV² < 0.49	sells almost every week, similar amounts	88
Erratic (spiky seller)	ADI < 1.32, CV² ≥ 0.49	sells most weeks, amounts swing	52
Intermittent (occasional)	ADI ≥ 1.32, CV² < 0.49	many zero weeks, steady amounts	284
Lumpy (unpredictable)	ADI ≥ 1.32, CV² ≥ 0.49	irregular weeks and amounts	175

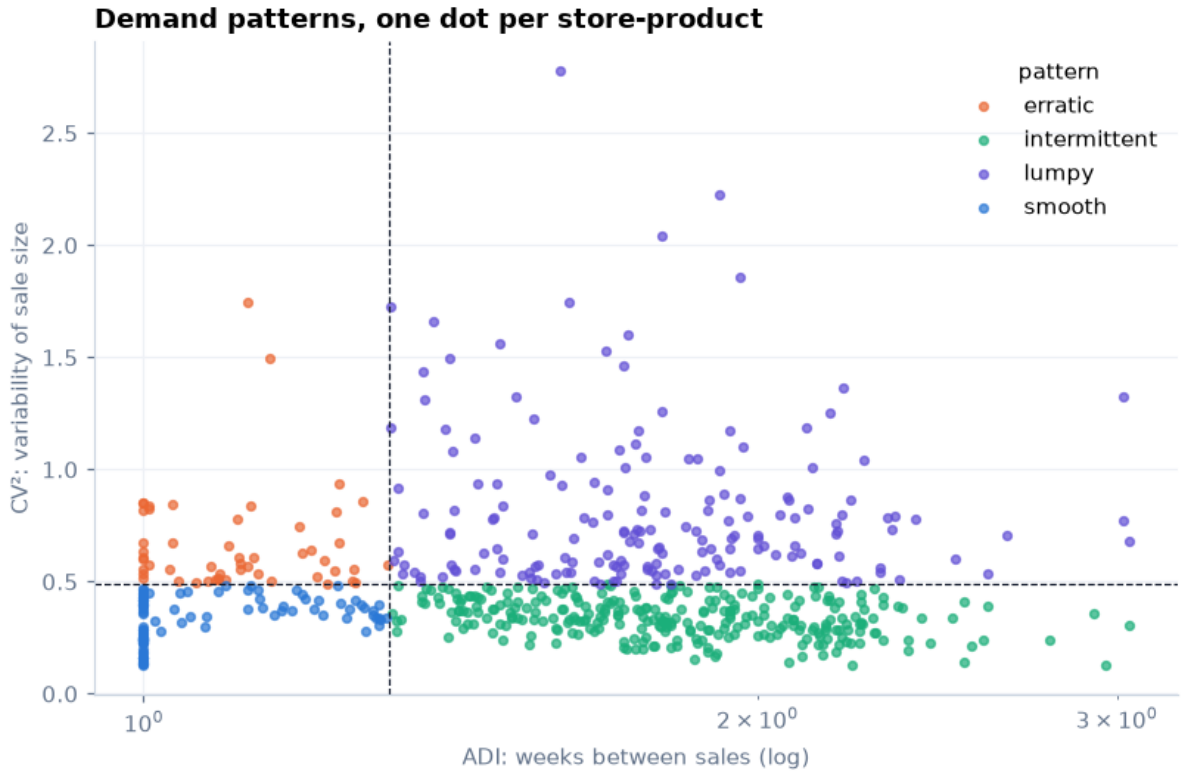


Figure 3. Demand patterns, one dot per store-product. Most are intermittent or lumpy: hard to forecast.

3.5 · The time split: no peeking at the future

Everything is evaluated as if it were happening live. The weeks are used in strictly separate roles:

Window	Weeks	Used for
Training history	up to each forecast date	fitting models (never later weeks)
Backtest dates 1-12	33 to 22 weeks before week 0	measuring the spread of forecast errors
Backtest dates 13-30	21 to 3 weeks before week 0	accuracy and tuning the ordering policy
Competition weeks 1-8	15 April to 3 June 2024	**final test only**: nothing is tuned on them

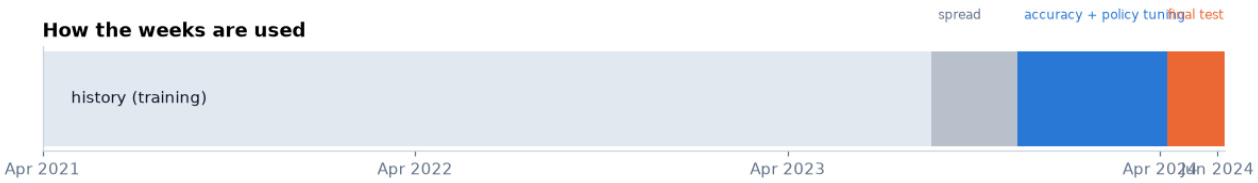


Figure 4. How the 165 weeks are used: history, the error-spread window, the accuracy and tuning window, and the final test.

PART 4

Exploratory data analysis

Exploratory data analysis (EDA) means looking at the data from many angles before modelling it, to understand its behaviour and choose suitable methods. Each graph below answers one question.

4.1 · The network over time (line graph)

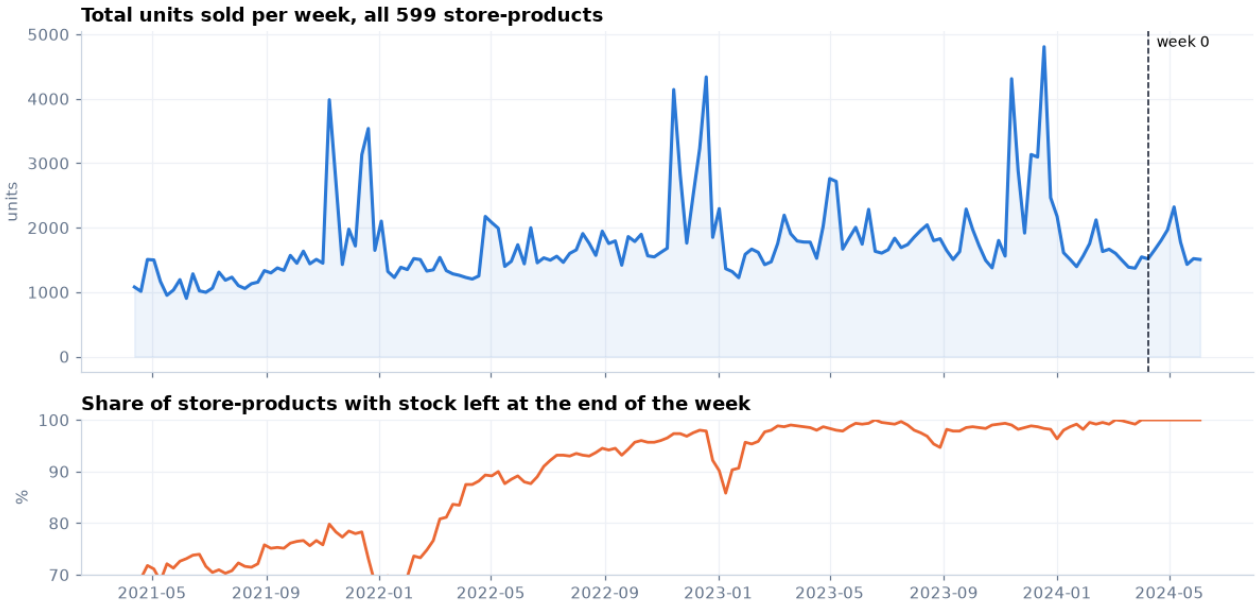


Figure 5. Top: total units sold per week by all 599 store-products (dashed line = week 0). Bottom: share of store-products with stock left at the end of each week.

The total sells about 1,761 units a week with a stable level and a sharp peak every December. Availability is not stable: in some periods 15-20% of store-products ended the week with an empty shelf.

4.2 · The seasonal profile

The seasonal factor of a calendar week is the average sales of that ISO week number divided by the average week (the official benchmark's own definition):

$$s_w = \text{mean sales in ISO week } w \div \text{mean of the 52 weekly means} \quad (1.0 = \text{an average week})$$

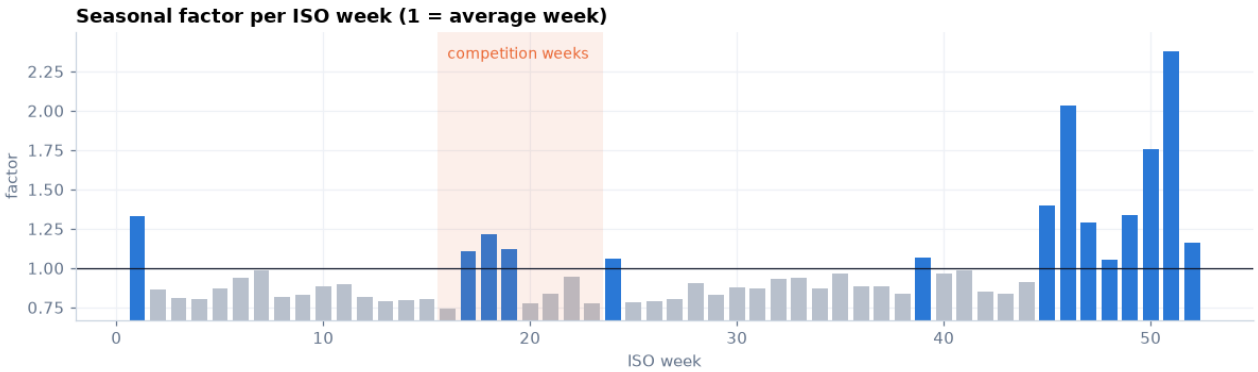


Figure 6. Seasonal factor per ISO week. Blue bars are above average. The shaded area marks the competition weeks (16-23).

Weeks 48-52 sell up to 2.4 times an average week (the Christmas peak). The competition weeks sit in a calm, slightly below-average part of the year.

4.3 · How much does a store-product sell? (histograms)

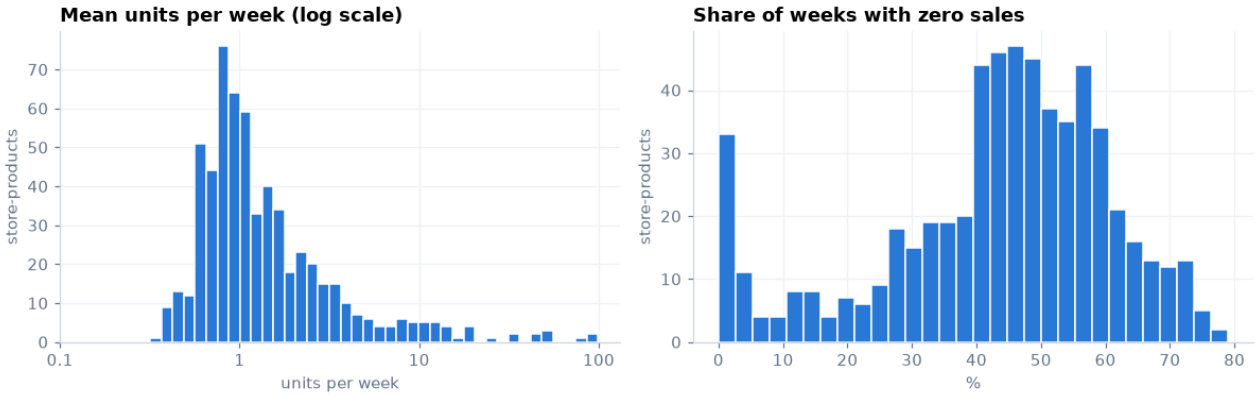


Figure 7. Left: average units per week per store-product (log scale). Right: share of zero-sales weeks per store-product.

The median store-product sells about 1.1 units a week, and 75% sell under 2 a week. Demand is extremely skewed: a few sell hundreds, most sell a handful. This is why error percentages look large later and why special slow-seller methods are needed.

4.4 · Where the volume is (Pareto curve)

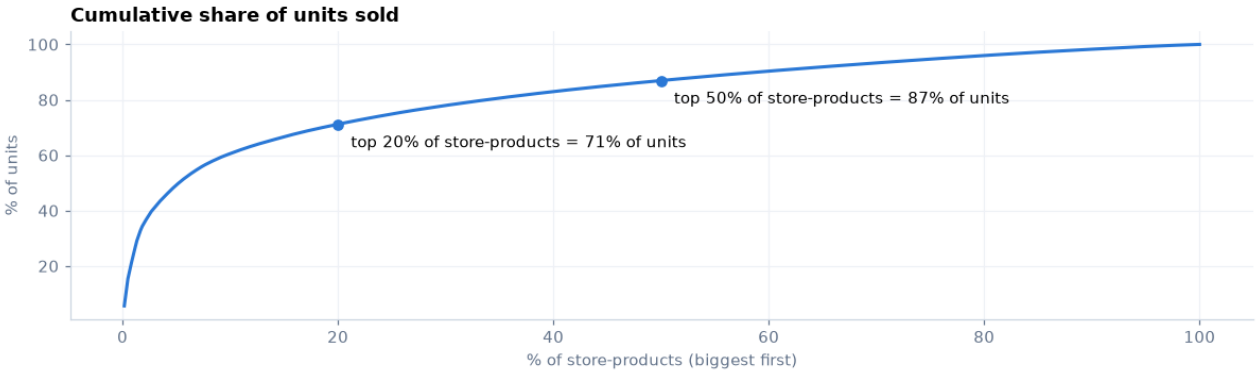


Figure 8. Cumulative share of units sold, store-products sorted from biggest to smallest seller.

A small group of store-products makes most of the units. Mistakes on those few items dominate the total cost.

4.5 · Scatter plots

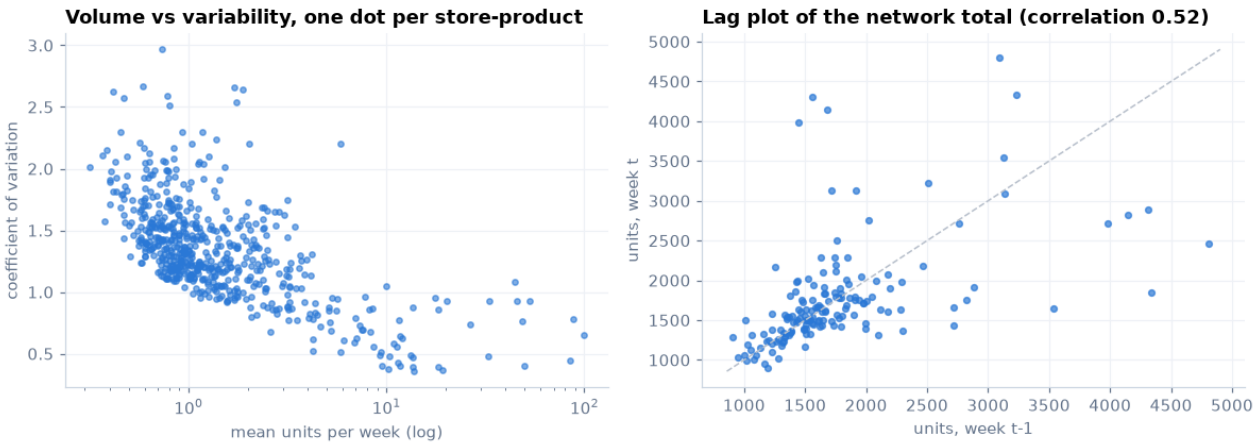


Figure 9. Left: volume against variability (coefficient of variation), one dot per store-product. Right: lag plot of the network total, this week against last week.

Slow sellers are the most variable relative to their size, typical of count data. The lag plot is tight around the diagonal: this week's total is strongly related to last week's, the property autoregressive (AR) models use.

4.6 · QQ plots (normality)

A QQ plot compares the quantiles of the data with those of a normal (bell-curve) distribution; points on the line mean normal.

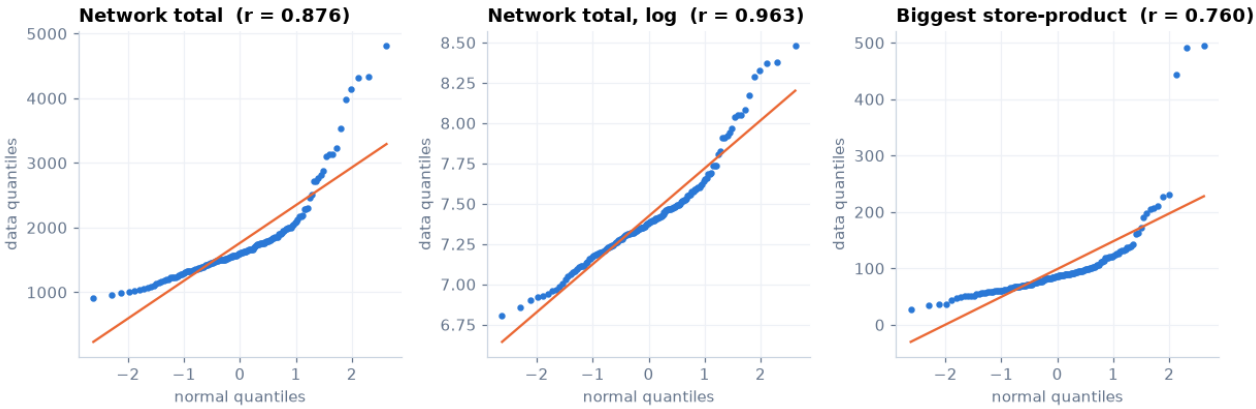


Figure 10. QQ plots of the network total, its logarithm, and the biggest single store-product.

The total is close to normal in the middle with a heavy upper tail (the Christmas weeks); the log helps. A single store-product is far from normal: small whole numbers with many zeros. Consequence: uncertainty is later modelled with a **count distribution** (negative binomial), not a bell curve.

4.7 · Descriptive statistics

For the network total, the biggest seller and the median store-product, and across all 599: mean, standard deviation, quartiles, coefficient of variation, skewness, excess kurtosis, share of zero weeks and the Jarque-Bera normality test. Results: strong right skew (skewness above 1), heavy tails (positive excess kurtosis), many zero weeks, and normality rejected almost everywhere.

4.8 · Stockouts and product groups

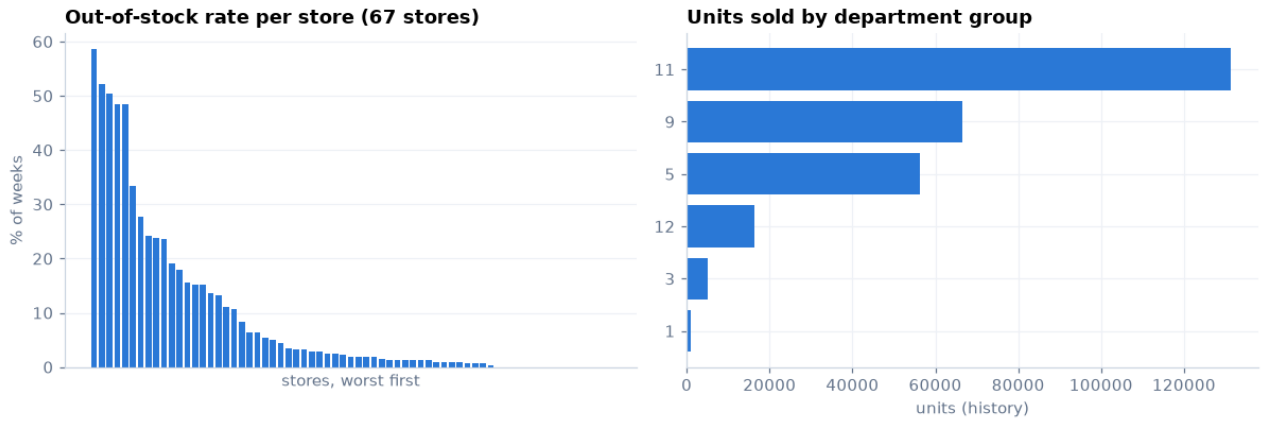


Figure 11. Left: out-of-stock rate per store, worst first. Right: units sold per department group.

Out-of-stock rates range from 0% to 59% of weeks across stores: empty shelves are a real, uneven problem.

What the EDA decided. (1) Slow, intermittent demand: include Croston and TSB and use a count distribution. (2) A strong yearly season shared across the network: take the season from the network. (3) Frequent censoring: learn only from clean weeks. (4) Store-products share structure: a single global machine-learning model can borrow strength across them.

PART 5

Time-series analysis (Box-Jenkins)

The Box-Jenkins method is the classical way to build a forecasting model for one series: check stationarity, transform if needed, read the autocorrelations to identify a model, choose its order with information criteria, then check that the residuals are white noise. It is applied to two series and then automatically to all 599:

- **The network total:** all 599 store-products added up, a smooth series of 157 weeks.
- **A representative store-product:** Store 61 · Product 124, a high-volume item sold from week 1, never out of stock (about 99 units a week).

5.1 · Stationarity: ADF and KPSS

A series is **stationary** when its mean, variance and correlations do not change over time. AR, MA and ARMA models assume it. The **Augmented Dickey-Fuller (ADF)** test fits:

$$\Delta y_t = \alpha + \gamma \cdot y_{t-1} + \delta_1 \Delta y_{t-1} + \ldots + \delta_p \Delta y_{t-p} + \varepsilon_t$$

H0: γ = 0 (a unit root, the series wanders like a random walk, not stationary). A p-value below 0.05 rejects H0: the series is stationary. The **KPSS** test reverses the question (H0: stationary), so the two together guard against a wrong conclusion. Results:

Series	ADF p-value	ADF says	KPSS says
Network total	0.006	stationary	not stationary (borderline)
Store 61 · Product 124	0.000	stationary	stationary
All 599 store-products	94% below 0.05	94% stationary	–

KPSS hesitates on the network total because of the strong yearly peaks; this is resolved below.

5.2 · Transformations

When a series is not stationary, standard fixes are: the **log** log(1 + y), which stabilises a variance that grows with the level; the **first difference** $y_t - y_{t-1}$, which removes a drifting level; and the **seasonal difference** $y_t - y_{t-52}$, which removes a repeating yearly pattern.

Transformation	ADF p	KPSS p	verdict
original	0.0065	0.023	borderline (tests disagree)
log(1+y)	0.0000	0.010	borderline (tests disagree)
1st difference	0.0000	0.100	stationary
log, 1st difference	0.0000	0.100	stationary
seasonal difference (52)	0.0022	0.010	borderline (tests disagree)

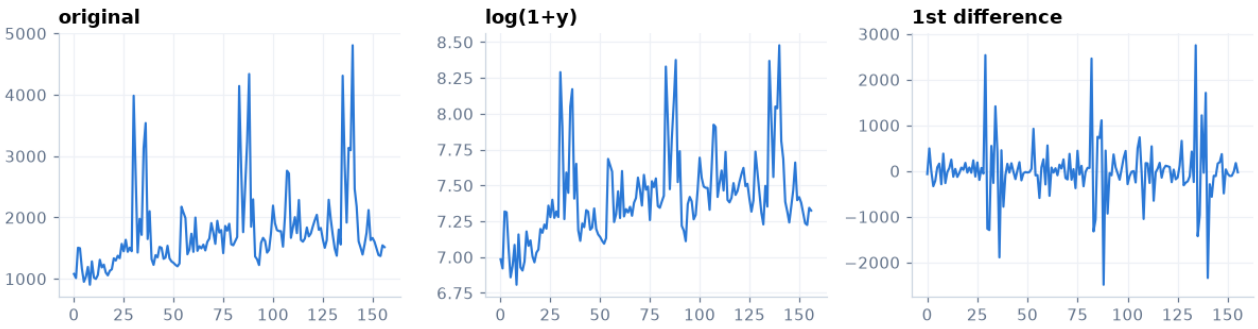


Figure 12. The network total: original, log and first difference.

The rule "difference only while ADF cannot reject a unit root" gives **d = 0**: no differencing is needed, so the original units are kept (orders need forecasts in units). The AIC comparison in 5.6 also tests differenced (d = 1) models as a double check.

5.3 · Seasonality: STL decomposition

STL (Seasonal-Trend decomposition using LOESS) splits the series into three additive parts with a 52-week season:

$$y_t = T_t \text{ (trend)} + S_t \text{ (season)} + R_t \text{ (remainder)}$$

$$\text{trend strength} = \max(0,\ 1 - \mathrm{Var}(R) \div \mathrm{Var}(T + R)) \qquad \text{seasonal strength} = \max(0,\ 1 - \mathrm{Var}(R) \div \mathrm{Var}(S + R))$$

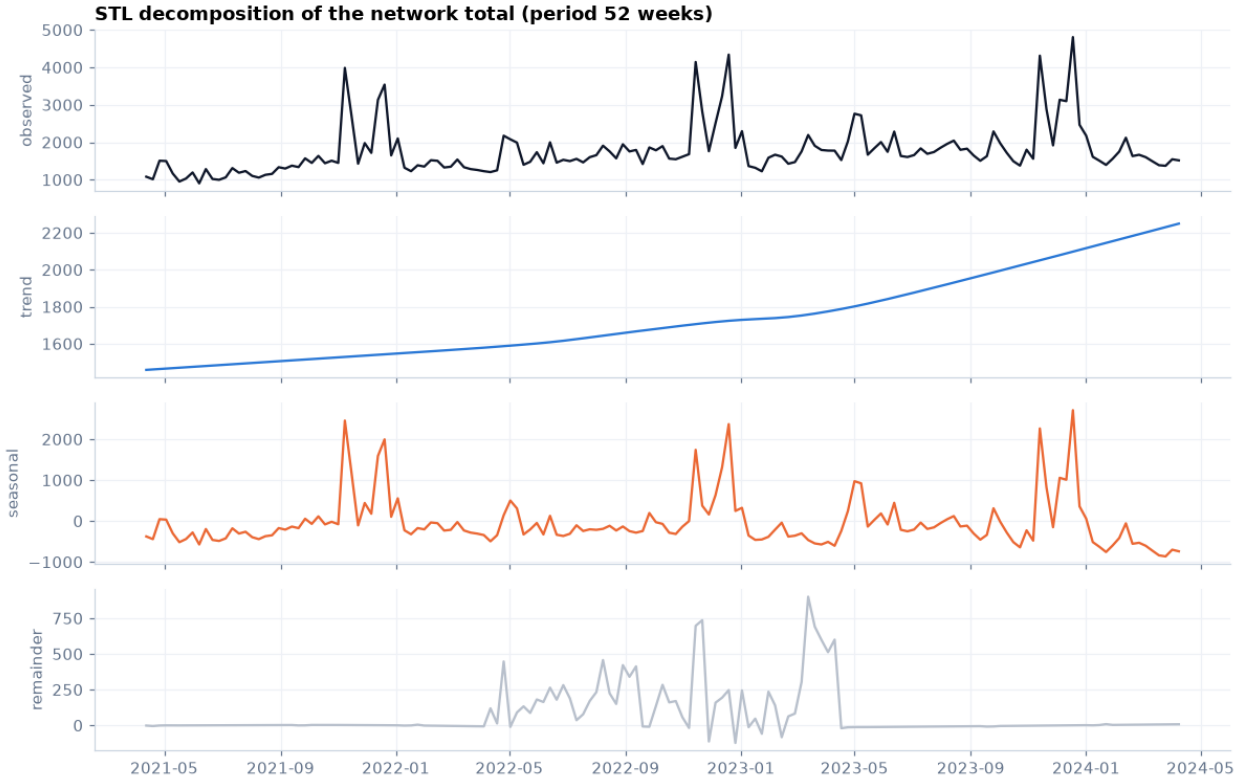


Figure 13. STL decomposition of the network total: observed, trend, yearly season and remainder.

Seasonal strength **0.93** and trend strength 0.61: the series is dominated by the yearly season. Single small store-products are too noisy and too short to estimate their own season, which is why the forecasting models take the season from the whole network.

5.4 · ACF and PACF

$$\text{ACF: } \rho_k = \Sigma(y_t - \bar{y})(y_{t-k} - \bar{y}) \div \Sigma(y_t - \bar{y})^2 \qquad \text{significance band} = \pm 1.96 \div \sqrt{n}$$

The **ACF** is the correlation of the series with itself k weeks earlier. The **PACF** is the same correlation after removing the effect of the weeks in between (the direct link only). Bars outside the band are significant at 5%.

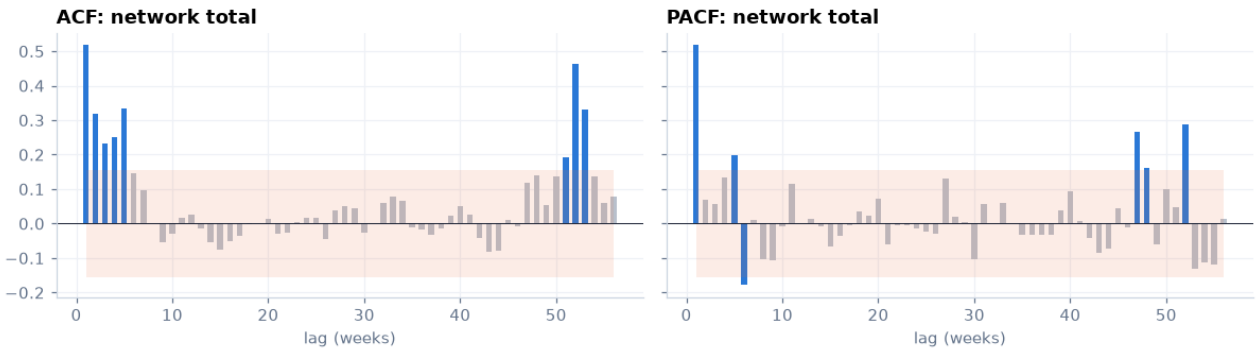


Figure 14. ACF and PACF of the network total (56 lags). Blue bars are significant.

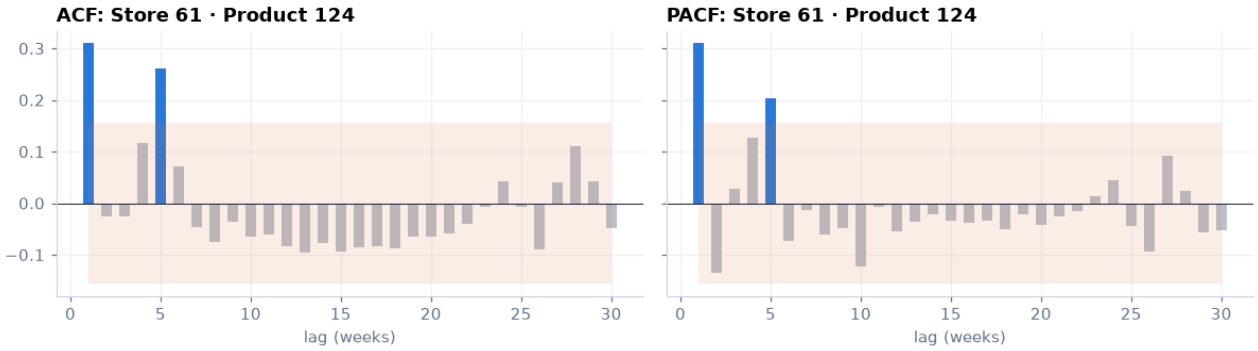


Figure 15. ACF and PACF of Store 61 · Product 124 (30 lags).

5.5 · Identifying the model and the lags

What the plots show	Model	Equation
PACF cuts off after p, ACF decays	AR(p)	$y_t = c + \varphi_1 y_{t-1} + \ldots + \varphi_p y_{t-p} + \varepsilon_t$
ACF cuts off after q, PACF decays	MA(q)	$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \ldots + \theta_q \varepsilon_{t-q}$
both decay	ARMA(p, q)	AR and MA terms together
differencing needed	ARIMA(p, d, q)	ARMA on the d-times differenced series
spike at the seasonal lag	SARIMA(p,d,q)(P,D,Q)[52]	adds terms at lags 52, 104, ...
nothing significant	white noise	no model can predict it

Network total: significant ACF lags [1, 2, 3, 4, 5, 51, 52, 53], PACF lags [1, 5, 6, 47, 48, 52]. The ACF decays and the PACF has one dominant spike at lag 1: an **AR** signature, plus the yearly lags around 52, so **SARIMA**. The representative store-product shows lags 1 and 5 only: a low-order ARMA. **Lags** 1-5 (recent weeks) and 52 (same week last year) carry the information; these also became the inputs of the machine-learning model.

5.6 · Choosing the order: AIC and BIC

$$AIC = 2k - 2 \cdot \ln L \qquad BIC = k \cdot \ln n - 2 \cdot \ln L \qquad (k = \text{parameters}, n = \text{weeks}, L = \text{likelihood})$$

Both reward a good fit (high likelihood) and penalise complexity (more parameters); BIC penalises harder. Lower is better; a difference below 2 means two models are practically equal. 27 candidates were fitted to the network total: ARMA(p, q) and SARIMA(p, 0, q)(1, 0, 0)[52] for p, q in 0-2, and ARIMA(p, 1, q).

Model	Family	AIC	BIC	ΔAIC
SARIMA(2, 0, 0)x(1, 0, 0)[52]	SARIMA	2379.9	2395.2	0.0
SARIMA(1, 0, 0)x(1, 0, 0)[52]	SARIMA	2380.4	2392.6	0.5
SARIMA(2, 0, 1)x(1, 0, 0)[52]	SARIMA	2382.3	2400.6	2.3
SARIMA(0, 0, 2)x(1, 0, 0)[52]	SARIMA	2387.1	2402.3	7.1
SARIMA(1, 0, 2)x(1, 0, 0)[52]	SARIMA	2388.7	2407.0	8.7
SARIMA(0, 0, 1)x(1, 0, 0)[52]	SARIMA	2389.3	2401.5	9.3
SARIMA(1, 0, 1)x(1, 0, 0)[52]	SARIMA	2391.3	2406.5	11.3
SARIMA(2, 0, 2)x(1, 0, 0)[52]	SARIMA	2398.0	2419.4	18.1

Best by AIC: **SARIMA(2, 0, 0)x(1, 0, 0)[52]**; best by BIC: SARIMA(1, 0, 0)x(1, 0, 0)[52] (one parameter fewer, ΔAIC 0.5). In words: this week's total depends on the last two weeks and on the same week one year earlier:

$$(1 - \varphi_1 B - \varphi_2 B^2)(1 - \Phi_1 B^{52})(y_t - \mu) = \varepsilon_t \qquad \varphi_1 = 0.32,\ \varphi_2 = 0.15,\ \Phi_1 = 0.70 \text{ (all significant)}$$

(B is the backshift operator: $B \cdot y_t = y_{t-1}$.)

5.7 · ARIMA for every store-product

The same recipe runs automatically on all 599 store-products: remove the yearly season with the network factors, choose d with the ADF rule, then choose (p, q) among 9 candidates by AIC. Store-products with fewer than 26 clean weeks get a mean-only model. Missing weeks stay missing for the Kalman filter.

Family chosen by AIC	Store-products	Share
ARMA	229	38%
mean only	166	28%
AR	119	20%
MA	53	9%
ARIMA	32	5%

Most store-products need no differencing and follow low-order AR, MA or ARMA models; 28% are slow sellers with so little signal that a constant mean wins.

5.8 · Residual diagnostics

The **residuals** are what the model could not explain. A good model leaves **white noise**: no pattern, constant variance.

Ljung-Box: $Q = n(n+2) \cdot \Sigma_{k=1..h}\, \hat{\rho}_k^{\,2} \div (n-k)$ (H0: no autocorrelation; p > 0.05 = white noise)

Jarque-Bera: $JB = n/6 \cdot (S^2 + (K-3)^2/4)$ (H0: normal residuals; S = skewness, K = kurtosis)

ARCH-LM: regress $\varepsilon_t^{\,2}$ on $\varepsilon_{t-1}^{\,2} \ldots \varepsilon_{t-q}^{\,2}$; $LM = n \cdot R^2$ (H0: constant variance; p < 0.05 = volatility clustering)

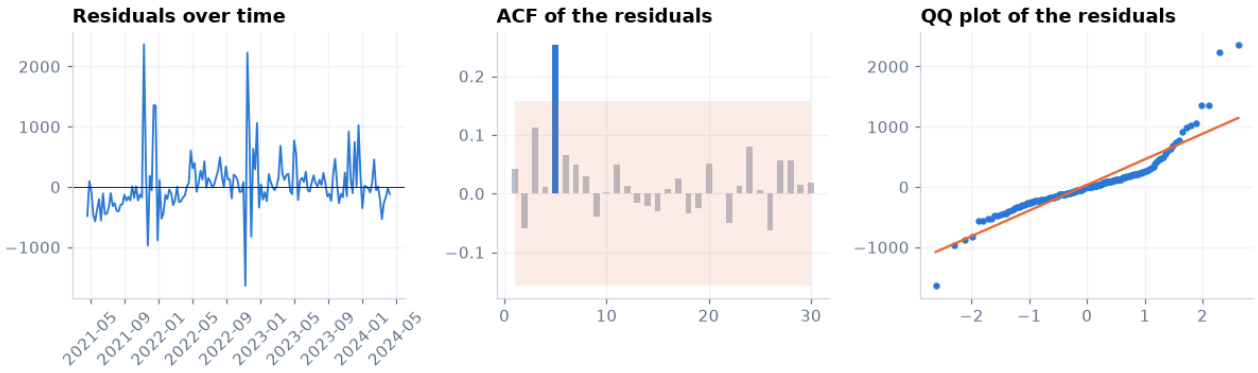


Figure 16. Residuals of the SARIMA on the network total: over time, their ACF, and their QQ plot.

5.9 · ARCH and GARCH

ARCH-LM found **volatility clustering** (p = 0.027): calm periods and turbulent periods (around Christmas) alternate, so the size of the error is itself predictable. A **GARCH(1, 1)** model describes this changing variance:

$$\varepsilon_t = \sigma_t \cdot z_t, \quad \sigma_t^{\,2} = \omega + \alpha \cdot \varepsilon_{t-1}^{\,2} + \beta \cdot \sigma_{t-1}^{\,2} \qquad \omega = 0.35,\ \alpha = 0.18,\ \beta = 0.45$$

α measures how strongly variance reacts to the last shock, β how long it remembers; α + β is the persistence. After GARCH, the **standardised residuals** $z_t = \varepsilon_t \div \sigma_t$ are tested again:

Test	SARIMA residuals	After GARCH(1,1)	Squared, after GARCH
Ljung-Box p (lag 4)	0.088	0.498	0.947
Ljung-Box p (lag 8)	0.012	0.051	0.747
Ljung-Box p (lag 13)	0.116	0.251	0.945
Ljung-Box p (lag 26)	0.697	0.742	1.000
Jarque-Bera p	0.000	0.000	0.000
ARCH-LM p	0.027	0.955	0.999
white noise	no	yes	yes
ARCH effect	yes	no	no

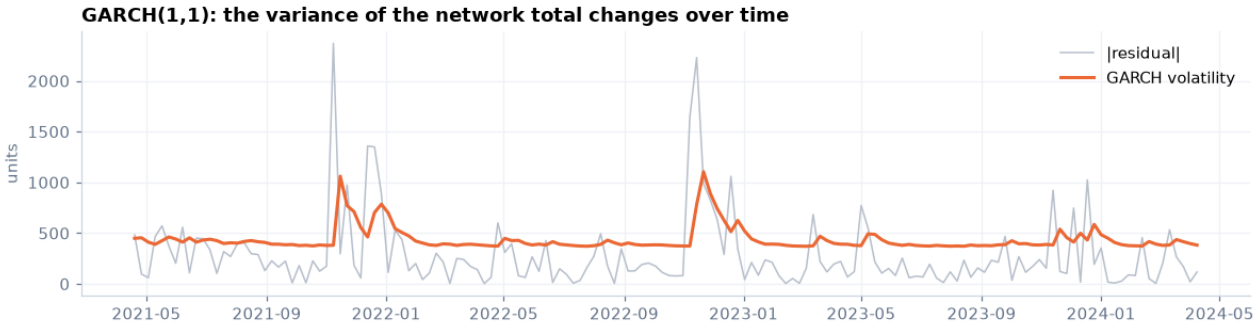


Figure 17. GARCH(1,1) volatility (orange) against the size of the residuals (grey).

After GARCH: no autocorrelation and no volatility clustering are left. Normality is still rejected because of a few extreme weeks, which does not bias the forecast but means normal-based intervals would be too narrow in those weeks.

5.10 · Does the model forecast unseen weeks?

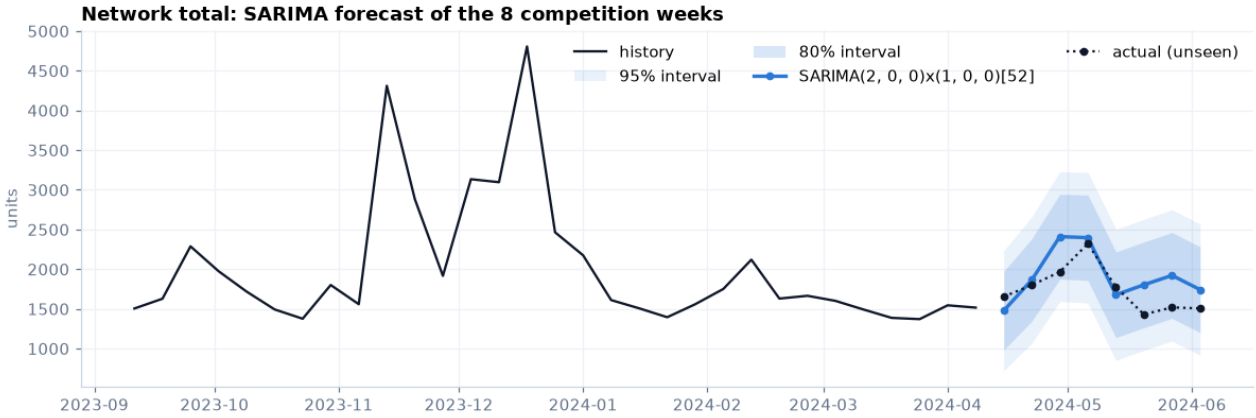


Figure 18. SARIMA forecast of the 8 competition weeks (made with history only), with 80% and 95% ranges, against what actually sold.

Mean absolute percentage error **14.2%**, against 20.0% for "same week last year"; **all 8 real weeks fall inside the 80% range**.

5.11 · The same tests on all 599 store-products

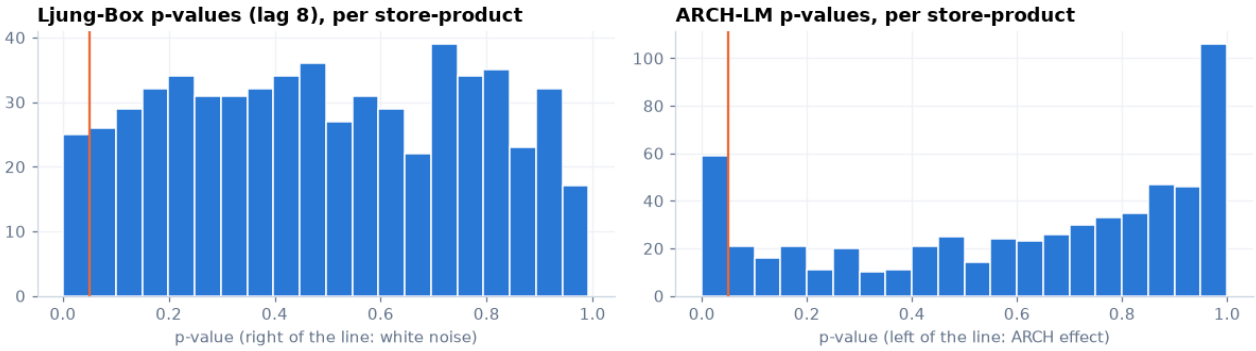


Figure 19. Ljung-Box (lag 8) and ARCH-LM p-values of the 599 per-product ARIMA models.

91% of store-products leave white-noise residuals (Ljung-Box at lags 4, 8 and 13). 10% show an ARCH effect; GARCH(1,1) removes it in 85% of them. With a 5% test level about 5% of perfectly good models fail by chance, so these numbers describe well-specified models.

Conclusion of the time-series analysis. The series are mostly stationary, strongly seasonal, and well described by low-order (S)ARIMA models whose residuals are white noise once GARCH absorbs the volatility clustering. The per-product ARIMA becomes one of the ten forecasting models.

PART 6

The ten forecasting models

Each model sees the clean demand up to the forecast date and predicts the next weeks (3 for the backtest, 4 for the competition rounds). *y* is clean demand, *ŷ* the forecast, *t* the last known week and *h* the number of weeks ahead. Ten models are compared so that the final choice is proven, not assumed.

6.1 · Last week (naive)

$$\hat{y}_{t+h} = \text{last clean value of } y$$

The simplest possible forecast. It reacts instantly but copies every random spike. It is the baseline any useful method must beat.

6.2 · 13-week average

$$\hat{y}_{t+h} = \text{mean of the clean weeks among the last 13}$$

A quarter of history smooths out noise but ignores the season and reacts slowly to change.

6.3 · Same week last year (seasonal naive)

$$\hat{y}_{t+h} = y_{t+h-52} \text{ (falls back to the 13-week average if that week is missing)}$$

Captures the season automatically, but repeats last year's noise as well.

6.4 · Official benchmark: seasonal moving average

$$\text{base} = \text{mean over the last 13 weeks of } (y_t \div s_{\text{week}(t)}) \quad \hat{y}_{t+h} = \text{base} \times s_{\text{week}(t+h)}$$

The competition's own method (vn2_benchmark.py), reproduced with the same arithmetic: out-of-stock weeks are set aside, sales are deseasonalised with the network factors s, averaged over 13 weeks, and reseasonalised for each future week. It is the target to beat.

6.5 · Exponential smoothing (SES on deseasonalised demand)

$$\ell_t = \ell_{t-1} + \alpha \cdot (y_t/s_t - \ell_{t-1}) \quad \hat{y}_{t+h} = \ell_t \times s_{t+h}$$

A weighted average whose weights fade geometrically into the past (weight α on the newest week, α(1−α) on the one before, ...). α is chosen per store-product among 0.05, 0.1, 0.2, 0.3 and 0.5 by the smallest one-step in-sample error. Missing weeks are skipped.

6.6 · ARIMA (AIC-selected, per store-product)

$$\varphi(B)(1 - B)^d(y_t/s_t) = c + \theta(B)\varepsilon_t \quad \hat{y} = \text{forecast} \times s_{t+h}$$

The Box-Jenkins model of Part 5: d by ADF, (p, q) by AIC, on season-adjusted clean demand. Orders and parameters are re-selected every 8 backtest dates (once for the competition) and run on newer data with the Kalman filter.

6.7 · Croston SBA (for intermittent demand)

$$\text{when a sale occurs: } z \leftarrow z + \alpha(y - z), \quad x \leftarrow x + \alpha(\text{gap} - x) \quad \hat{y} = (1 - \alpha/2) \cdot z \div x, \quad \alpha = 0.1$$

Croston's idea: for slow sellers, predict separately **how big a sale is** (z) and **how many weeks pass between sales** (x); demand per week is z ÷ x. The factor (1 − α/2) is the Syntetos-Boylan correction of Croston's known upward bias.

6.8 · TSB (Teunter-Syntetos-Babai)

$$\text{every week: } p \leftarrow p + \beta(1_{\text{sale}} - p); \quad \text{on a sale: } z \leftarrow z + \alpha(y - z) \quad \hat{y} = p \cdot z, \quad \alpha = \beta = 0.1$$

Like Croston but smoothing the **probability of a sale** p every week, so the forecast decays when a product stops selling (Croston would keep forecasting the old rate).

6.9 · LightGBM (global machine learning)

One gradient-boosted decision-tree model is trained on all 599 store-products at once. Gradient boosting adds trees one after another, each correcting the errors of the previous ones:

$$F(x) = f_1(x) + f_2(x) + \ldots + f_{300}(x), \quad \text{each } f \text{ a small decision tree}$$

Target: clean demand h weeks ahead divided by the store-product's recent level (13-week mean + 0.1), so slow and fast sellers share one model; the prediction is multiplied back by the level. **Horizon** is an input, so one model serves 1, 2, 3 and 4 weeks ahead. Out-of-stock weeks are never used as targets.

Feature group	Features (all known at the forecast date)
Recent demand	clean demand 1-8 weeks ago; raw sales 1-4 weeks ago; in-stock flags 1-4 weeks ago
Rolling statistics	means over 4, 8, 26 and 52 weeks; 13-week standard deviation; share of zero weeks; in-stock share
Season and calendar	same week last year and two years ago; seasonal factor of the target week; target/now season ratio; ISO week; horizon
Product life	weeks since launch; log of the level
Hierarchy (categorical)	Store, Product, ProductGroup, Division, Department, DepartmentGroup, StoreFormat, Format

Loss function: Tweedie (power 1.5), designed for non-negative data with many exact zeros. In a test on past weeks it beat squared error, absolute error and Huber loss. **Settings:** 300 trees, learning rate 0.05, 31 leaves, at least 40 rows per leaf, 80% row and column sampling, L2 penalty 1, rows weighted by the level. Refitted every 4 backtest dates and every competition round.

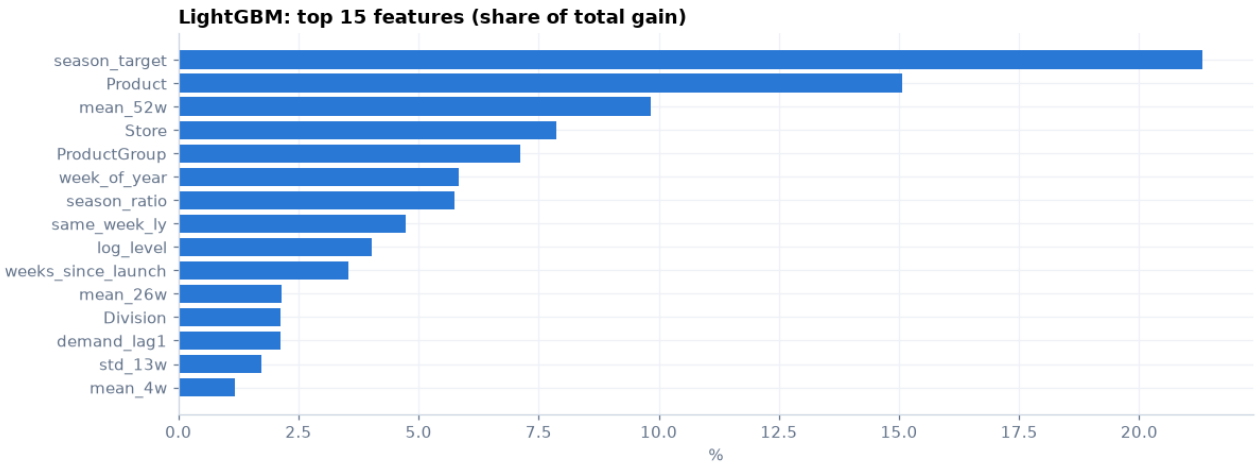


Figure 20. LightGBM: the 15 most important features (share of total gain).

The target week's seasonal factor and the product's identity lead, then the yearly average and the store: the model learns both the calendar and each item's typical level.

6.10 · Ensemble

$$\hat{y} = (\hat{y}_{\text{LightGBM}} + \hat{y}_{\text{exponential smoothing}} + \hat{y}_{\text{official benchmark}}) \div 3$$

Averaging different kinds of models cancels part of their individual errors; it is one of the most reliable findings in forecasting research.

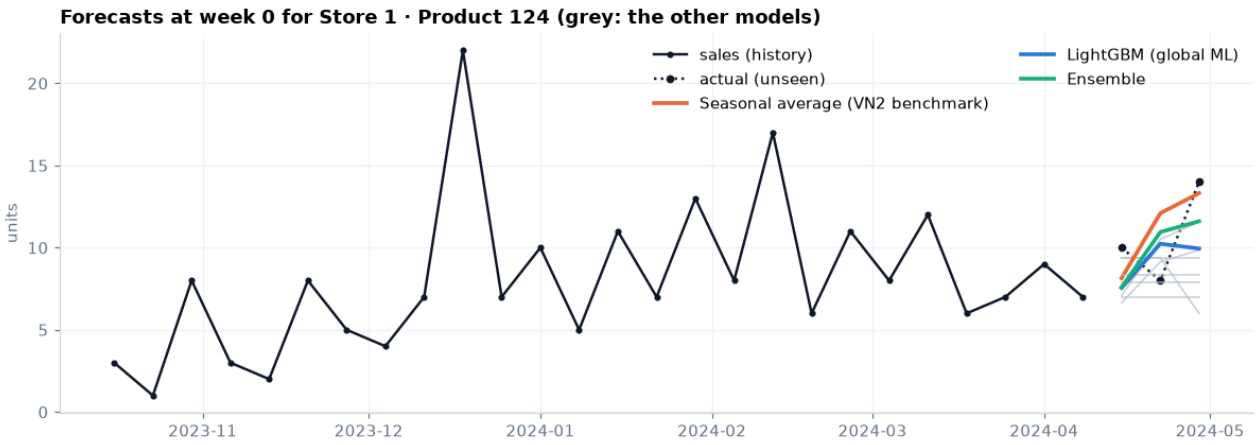


Figure 21. All models' forecasts at week 0 for one store-product (three highlighted, seven in grey), with the history and what then sold.

PART 7

Testing the forecasts: the backtest

7.1 · Design

A **rolling-origin backtest**: at each of 30 past weeks (the origins), every model is given only the data up to that week and forecasts the next 3 weeks; the forecasts are then compared with what really sold. Using any later data would be *leakage* and make results look better than reality. Three weeks because an order placed now arrives in two weeks: the planner needs this week, next week and the arrival week. Errors are scored only on clean weeks.

7.2 · Error measures

$$e = \hat{y} - y \quad MAE = \text{mean}|e| \quad WAPE = \Sigma|e| \div \Sigma y \quad Bias = \Sigma e \div \Sigma y \quad Score = WAPE + |Bias|$$

MAE is the average miss in units. WAPE scales the total error by total demand. Bias shows whether forecasts are too high (+) or too low (−) overall. **Score**, the official metric of the VN1 forecasting competition, punishes both noise and systematic bias.

7.3 · Results

Model	MAE	RMSE	Bias %	WAPE %	Score %
LightGBM	1.89	6.29	-1.9	55.2	57.1
Ensemble	1.96	6.45	2.4	57.2	59.6
Official benchmark (seasonal average)	2.06	6.85	3.5	60.1	63.7
TSB	2.17	8.18	0.9	63.4	64.3
ARIMA (AIC-selected)	2.07	6.81	4.6	60.4	65.0
Croston SBA	2.15	8.19	-2.9	62.7	65.6
Exponential smoothing	2.07	7.04	5.6	60.4	66.1
13-week average	2.27	8.48	3.2	66.2	69.4
Same week last year	2.41	7.11	-0.5	70.3	70.8
Last week	2.55	9.62	1.0	74.6	75.6

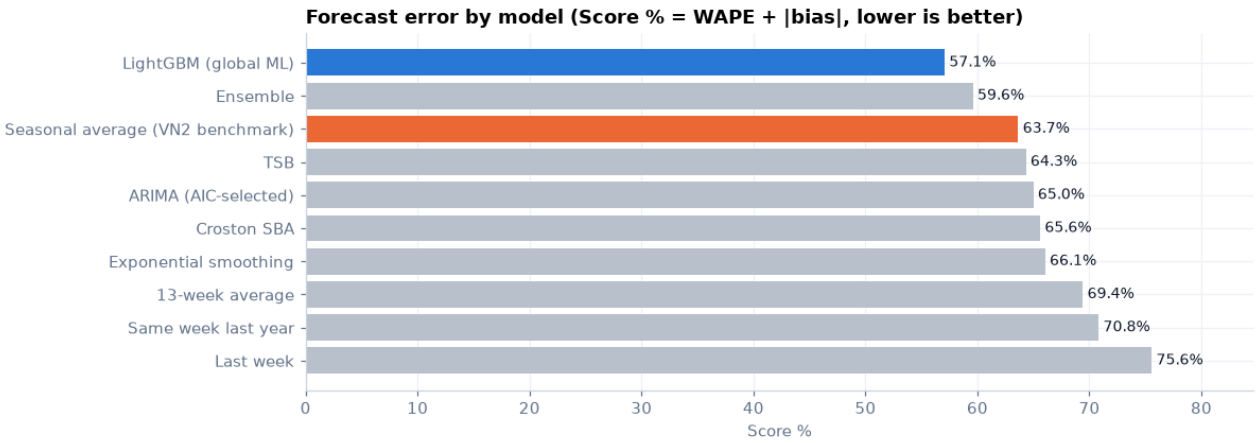


Figure 22. Score % by model (lower is better). LightGBM in blue, the official benchmark in orange.

Why are errors around 60%? Most store-products sell one or two units: forecasting 2 when 1 sells is a 100% error. Percentages for slow sellers always look large; what matters is the comparison between models and, later, the cost.

By horizon (Score %)

Model	1 week ahead	2 weeks	3 weeks
LightGBM	55.2	57.3	58.8
Ensemble	58.9	59.7	60.3
Official benchmark (seasonal average)	63.0	63.8	64.1
TSB	62.9	64.7	65.5
ARIMA (AIC-selected)	64.4	65.4	65.2
Croston SBA	64.7	65.8	66.4
Exponential smoothing	64.5	66.7	66.9
13-week average	67.6	69.8	70.9
Same week last year	70.9	70.9	70.7
Last week	70.3	76.4	80.0

By demand pattern (Score %)

Model	smooth	erratic	intermittent	lumpy
LightGBM	34.8	62.9	104.6	96.1
Ensemble	38.1	56.2	109.7	103.9
Official benchmark (seasonal average)	41.9	56.7	113.5	109.4
TSB	41.0	66.4	111.0	111.2
ARIMA (AIC-selected)	44.9	62.9	127.4	114.0
Croston SBA	44.7	70.1	111.1	108.7
Exponential smoothing	43.2	56.8	114.7	113.0
13-week average	43.3	66.1	112.8	119.4
Same week last year	40.9	92.2	148.8	136.9
Last week	52.3	67.9	118.1	126.6

Forecasting gets slightly harder further ahead and much harder for intermittent and lumpy store-products, where errors above 100% are common. LightGBM leads in every pattern except erratic.

7.4 · Are the forecast errors white noise?

For each model and store-product, the sequence of 1-week-ahead errors over the 30 dates is tested with Ljung-Box (lag 4). Passing means the model leaves no pattern in its mistakes.

Model	Store-products tested	White-noise share	Median p
Exponential smoothing	599	93%	0.44
Ensemble	599	93%	0.46
LightGBM	599	92%	0.45
TSB	599	92%	0.46
Croston SBA	599	91%	0.42
Same week last year	599	90%	0.44
Official benchmark (seasonal average)	599	90%	0.43
ARIMA (AIC-selected)	599	89%	0.40
13-week average	599	88%	0.39
Last week	599	52%	0.06

A Ljung-Box pass shows that no exploitable pattern is left; it does not by itself prove accuracy (a flat average can pass). Accuracy (7.3) and white-noise errors (7.4) together are the evidence.

PART 8

Uncertainty: how wrong can a forecast be?

Ordering exactly the forecast would run out about half the time. The policy needs the whole **distribution** of demand around each forecast. Weekly unit counts are modelled with a **negative binomial** distribution, built for counts with extra spread:

$$\text{mean} = \mu \text{ (the forecast)}, \quad \text{variance} = k \cdot \mu \quad (k = 1\text{: Poisson noise; } k > 1\text{: spikier})$$

$$\text{Demand} = \text{Poisson}(\lambda) \text{ with } \lambda \sim \text{Gamma}(\text{shape } r, \text{mean } \mu), \quad r = \mu \div (k - 1)$$

Estimating k. For each model and store-product, from the backtest errors on clean weeks:

$$\text{raw } k = \Sigma(\hat{y} - y)^2 \div \Sigma\hat{y} \quad k = w \cdot \text{raw} + (1 - w) \cdot \text{pooled } k, \quad w = n \div (n + 6), \quad \text{kept in } [1, 30]$$

Each store-product has few clean weeks, so its own estimate is shrunk towards the model's pooled value. To keep the test honest, k for the policy tuning comes only from backtest dates 1-12; dates 13-30 check calibration: demand should stay below the 50%, 80% and 95% quantiles about 50%, 80% and 95% of the time.

Model	pooled k	demand ≤ q50	demand ≤ q80	demand ≤ q95
ARIMA (AIC-selected)	17.7	51%	86%	98%
Croston SBA	29.1	46%	83%	98%
Ensemble	15.9	51%	86%	99%
LightGBM (global ML)	15.1	51%	85%	98%
13-week average	27.9	50%	83%	98%
Last week	36.8	48%	71%	81%
Seasonal average (VN2 benchmark)	16.8	50%	83%	98%
Exponential smoothing	17.3	51%	85%	98%
Same week last year	13.0	55%	76%	83%
TSB	27.1	48%	84%	98%

The quantiles are close but slightly generous (about 85% instead of 80%): the spread is estimated mostly from large, volatile items. This is corrected in Part 10 by tuning the service level with real simulated costs.

PART 9

The inventory simulator

The simulator plays the official rules (the organiser's vn2_simulate.py) for every store-product, every week:

$$\text{start} = \text{end}_{\text{last week}} + \text{arriving order} \quad \text{sales} = \min(\text{start}, \text{demand}) \quad \text{lost} = \text{demand} - \text{sales}$$

$$\text{end} = \text{start} - \text{sales} \quad \text{cost} = 0.2 \times \text{end} + 1.0 \times \text{lost} \quad \text{in transit: W+1} \leftarrow \text{W+2, W+2} \leftarrow \text{new order}$$

Demand in the simulation is the recorded sales, as in the competition. The state of each store-product is three numbers: stock on the shelf, the order arriving next week, and the order arriving in two weeks.

9.1 · Verification against the official result

The organiser published the six benchmark order files and their score. Playing them through our simulator, and re-implementing the benchmark policy (seasonal average, order up to 4 weeks of forecast), must give the same number:

Check	Result
Setup cost, weeks 1-2	€913.80 (official: €913.80)
Score of the published benchmark orders, weeks 3-8	€4,334.00 (official: €4,334.00)
Our re-implementation of the benchmark	€4,334.00, identical orders for all 599 × 6 decisions

Week	Start	Sales	Lost	End	Holding	Shortage	Cost
1.0	2,225.0	1,432.0	222.0	793.0	158.6	222.0	380.6
2.0	2,492.0	1,471.0	329.0	1,021.0	204.2	329.0	533.2
3.0	5,008.0	1,819.0	147.0	3,189.0	637.8	147.0	784.8
4.0	4,846.0	1,979.0	342.0	2,867.0	573.4	342.0	915.4
5.0	3,910.0	1,519.0	255.0	2,391.0	478.2	255.0	733.2
6.0	3,750.0	1,314.0	116.0	2,436.0	487.2	116.0	603.2
7.0	3,837.0	1,402.0	117.0	2,435.0	487.0	117.0	604.0
8.0	4,367.0	1,405.0	101.0	2,962.0	592.4	101.0	693.4

The official benchmark, week by week (all 599 store-products).

> Because the simulator reproduces the official result exactly, every score in this project is directly comparable with the real competition leaderboard.

PART 10

The ordering policy

10.1 · The idea: a newsvendor problem

Before week k the planner orders Q units that land at the start of week k + 2. When they land, the shelf holds X, whatever is left after weeks k and k + 1 (with lost sales). The order must cover Y = demand in week k + 2 − X. With cost h per unit left and p per unit short, the expected cost of the arrival week is:

$$E[\text{cost}] = h \cdot E[(Q - Y)^{+}] + p \cdot E[(Y - Q)^{+}]$$

The classic newsvendor result: the best Q is the **critical-ratio quantile** of Y:

$$Q^{*} = F_{Y}^{-1}(p \div (p + h)) = F_{Y}^{-1}(1 \div 1.2) = F_{Y}^{-1}(0.833)$$

The probability used is called the **service level**: the chance that the order covers its week.

10.2 · Monte Carlo: computing the distribution of Y

- Draw 1,000 demand paths for weeks k, k+1 and k+2 from the negative binomial (forecast μ, spread k).
- On each path, play weeks k and k+1 with the stock on the shelf and the two orders on their way, with lost sales: $X = \max(0, \max(0, \text{shelf} + \text{arriving}_{k} - D_{k}) + \text{arriving}_{k+1} - D_{k+1})$.
- $Y = D_{k+2} - X$ on each path; Q = the service-level quantile of the 1,000 values of Y, at least 0, a whole number.
- Common random numbers: all service levels share the same paths, so differences come from the policy alone.

10.3 · Tuning the service level by simulation

The textbook 0.833 assumes a perfect demand distribution. Instead, every model is replayed over 18 past weeks (backtest dates 13-30) at 12 service levels; the first 3 weeks are warm-up and not scored. The benchmark's simpler order-up-to-N-weeks rule is replayed too.

Service level	Ensemble	LightGBM	Official benchmark	ARIMA	Exponential smoothing	TSB
0.500	14,698	14,619	15,178	15,091	14,978	15,669
0.600	13,108	13,030	13,708	13,515	13,436	14,380
0.700	11,740	11,800	12,492	12,484	12,350	13,496
0.750	11,435	11,467	12,195	12,343	12,158	13,526
0.800	11,527	11,509	12,324	12,721	12,490	14,143
0.833	11,969	11,928	12,796	13,360	13,144	15,013
0.850	12,364	12,289	13,202	13,852	13,648	15,702
0.880	13,449	13,293	14,291	15,067	14,963	17,343
0.900	14,475	14,317	15,386	16,257	16,260	18,865
0.920	15,902	15,752	16,884	17,806	17,981	20,902
0.940	17,907	17,771	19,033	20,005	20,416	23,679

Service level	Ensemble	LightGBM	Official benchmark	ARIMA	Exponential smoothing	TSB
0.960	21,154	20,810	22,378	23,334	23,937	27,806

Historical replay: total cost (€) over 15 scored weeks by service level, for six models.

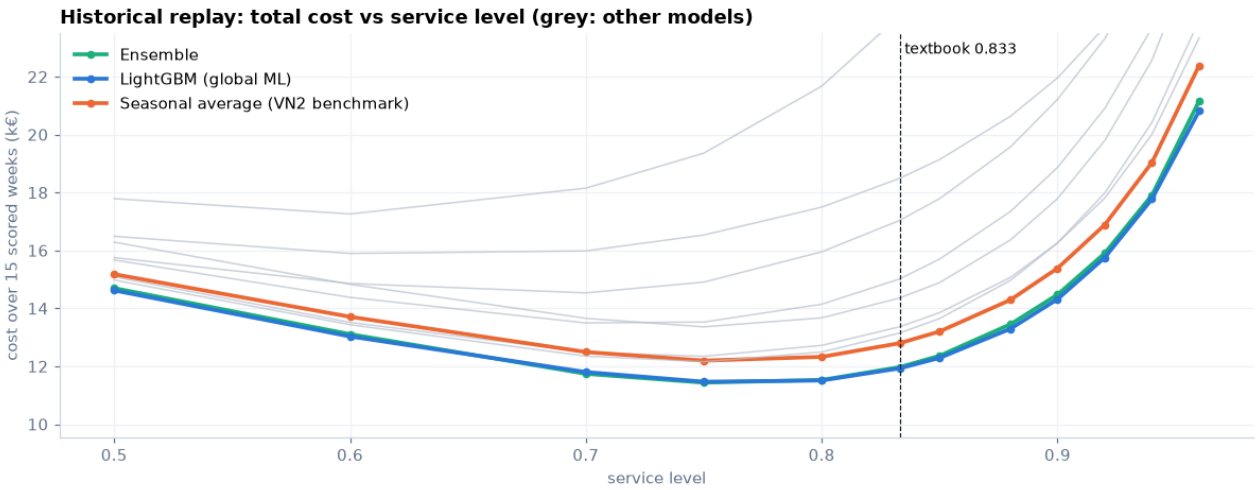


Figure 23. Replay cost against service level (grey: other models). The dashed line is the textbook 0.833.

Every curve is U-shaped: too little safety means empty shelves, too much means leftovers. The lowest cost is at **0.75** for the best models, below the textbook 0.833, for two reasons: the estimated spread is slightly generous (Part 8), and leftover stock is not wasted (it sells the following week). The chosen plan is the cheapest in the replay: **Ensemble at 0.75**. Simulation-based tuning is exactly what the competition's top teams reported doing.

PART 11

The final test: the eight competition weeks

The competition is replayed exactly: before each round the models are refitted on the data available then (sales and in-stock flags up to the previous week) and forecast 4 weeks ahead; each model orders with the service level chosen on the past. **Nothing is tuned on these weeks.**

11.1 · Forecast accuracy on the competition weeks

Model	MAE	Bias %	WAPE %	Score %
Official benchmark (seasonal average)	1.60	2.0	54.6	56.7
Ensemble	1.58	4.5	54.1	58.6
Croston SBA	1.67	2.0	57.2	59.2
TSB	1.65	3.9	56.4	60.3
13-week average	1.60	-6.2	54.7	60.8
LightGBM	1.58	6.8	54.2	60.9
Exponential smoothing	1.64	4.7	56.2	60.9
ARIMA (AIC-selected)	1.67	5.8	57.1	62.9
Last week	1.99	1.8	68.0	69.9
Same week last year	2.36	19.7	80.5	100.3

An honest surprise: over these eight weeks the simple official benchmark forecast is the most accurate, and LightGBM (best over 30 backtest weeks) slips. Eight weeks is a short window and rankings move. What matters is the cost of the orders each forecast leads to:

11.2 · The score

Model	Service level	Newsvendor score	Order-up-to score	Hindsight best
Ensemble	0.75	€3,626.4	€3,798.4	€3,626.4
LightGBM	0.75	€3,680.2	€3,802.2	€3,680.2
Official benchmark (seasonal average)	0.75	€3,750.8	€3,901.0	€3,736.8
ARIMA (AIC-selected)	0.75	€3,756.8	€3,993.8	€3,756.8
Exponential smoothing	0.75	€3,817.0	€4,107.0	€3,817.0
Croston SBA	0.75	€3,976.8	€4,041.0	€3,905.0
TSB	0.70	€4,009.2	€4,433.0	€3,836.2
13-week average	0.70	€4,277.2	€4,805.4	€3,910.4
Last week	0.60	€5,218.4	€6,039.6	€5,059.4
Same week last year	0.60	€5,468.4	€5,458.6	€5,468.4

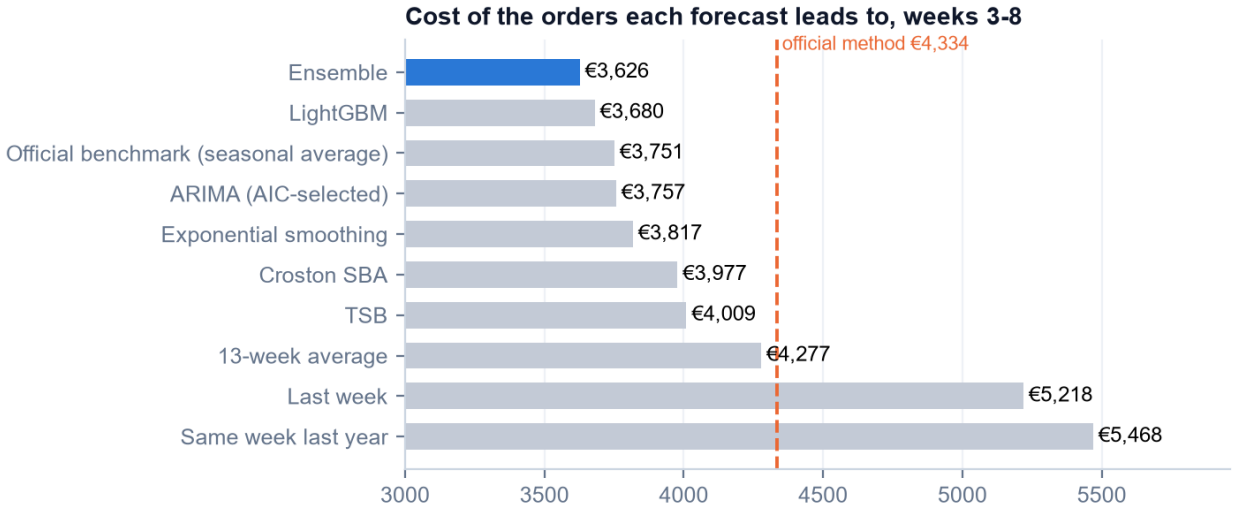


Figure 24. Cost of weeks 3-8 for each model's orders; the dashed line is the official method.

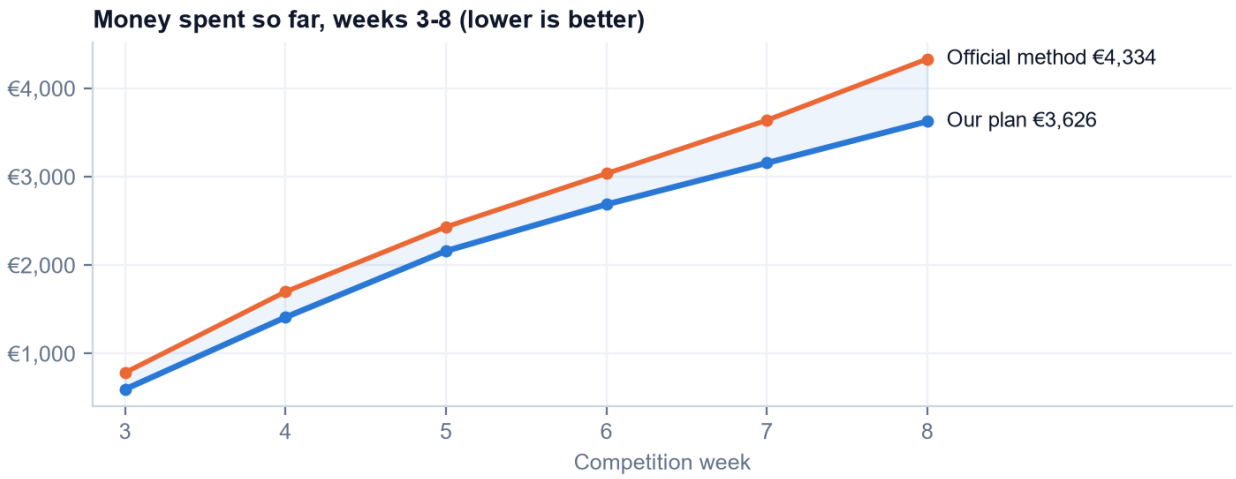


Figure 25. Money spent so far, weeks 3-8: our plan (blue) against the official method (orange).

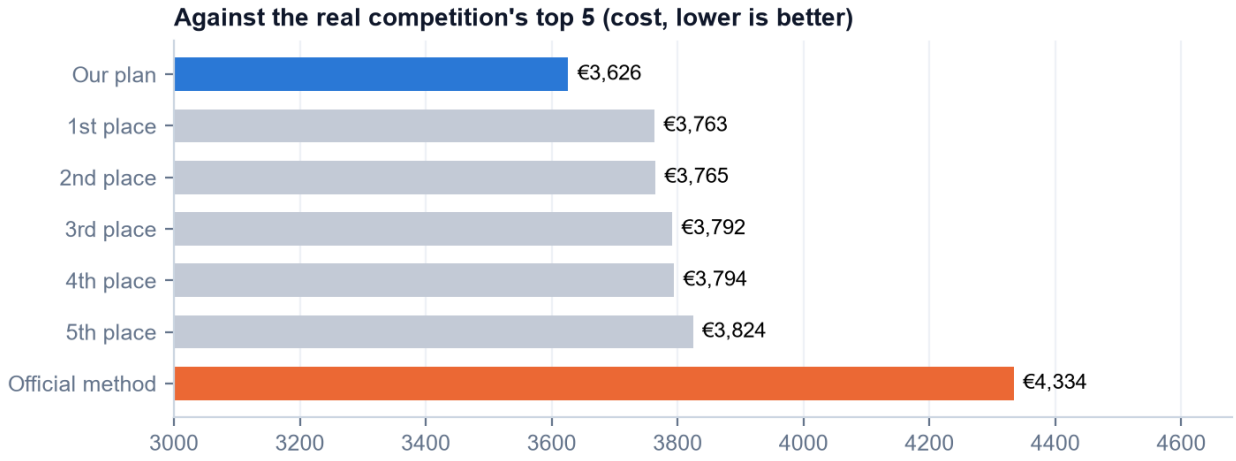


Figure 26. Our plan against the real competition's top five and the official method.

Plan	Cost, weeks 3-8
Our plan: Ensemble @ 0.75	**€3,626.4**
Real 1st place	€3,763.2
Official benchmark	**€4,334.0**
Saving vs the official benchmark	€707.6 (16.3%)

11.3 · Where the saving comes from

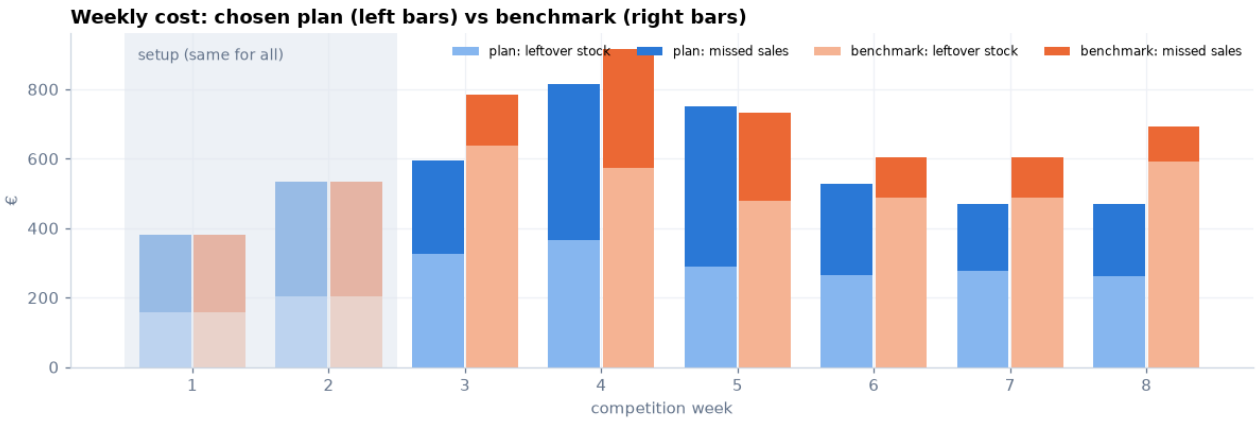


Figure 27. Weekly cost: our plan (left bars: light = leftover stock, dark = missed sales) against the benchmark (right bars).

Our plan keeps **45% less stock** on the shelves (1,487 vs 2,713 units at the end of an average week) and accepts more missed sales (1,842 vs 1,078 units over weeks 3-8). Under the official costs that is the cheaper deal: 1,226 fewer units held each week save about €1,472 over the six weeks, more than the €764 of extra missed sales.

How to read this honestly. Every decision used only information available at the time, and the service level was chosen on past weeks. But the project was built after the competition, knowing which kinds of methods did well (global gradient boosting, simulation-tuned policies). The fair claim is: the pipeline clearly beats the official method (16% cheaper) and performs in the range of the top entries. "We beat 1st place" is not a fair claim.

PART 12

The stock outlook: the next order

The data ends on 3 June 2024 (week 8). The plan treats **10 June 2024 as "today"**: it continues the plan for rounds 7 and 8 (which the competition did not play), then forecasts weeks 9-12 from all data and computes the next order, which arrives on 24 June (week 11). Today's real date cannot be used: there is no data after June 2024, so a plan for today would be invented.

Outlook figure	Value
Units to order now	1,162
Store-products receiving an order	271 of 599
High risk (> 50%) of an empty shelf in week 9	19
High risk in week 10	6
High risk in week 11 (after this order arrives)	0

Weeks 9 and 10 are already decided: only the stock on the shelf and the orders on their way can serve them. The new order protects week 11, which is why its risk is close to 1 − service level for most store-products. The risk of a week is the share of the 1,000 simulated demand paths in which the shelf runs empty.

PART 13

The dashboard, page by page

The dashboard (Streamlit, *streamlit run app/Home.py*, http://localhost:8501) has seven pages. Heavy results come from the notebook's artefacts; the stock plan is re-simulated live when a setting changes. Colours are consistent everywhere: **blue = our plan**, **orange = the official method**, grey = context, and the palette is colour-blind safe. Every chart has a plain-language note.

Overview: did it work?

- Hero header: the business problem in two lines.
- Money headlines: saved vs the official method, the yearly estimate, stock on shelves, units to order this week, and an honest note on the trade-off (more missed sales, much less stock).
- Meet the products: five hero store-products chosen by fixed rules (The Steady Seller, The Christmas Rocket, The Often-Empty Shelf, The Slow Mover, The Newcomer), each with its real code, a sparkline and a Follow button.
- The story: four charts for the followed product (it sold like this, we predicted this, so we ordered this, and the shelf ...), with one plain sentence each, including when the official method did better on that product.
- The result: money spent so far, our plan vs the official method; the real competition's top five.
- How it works (four steps) and links to the other pages.

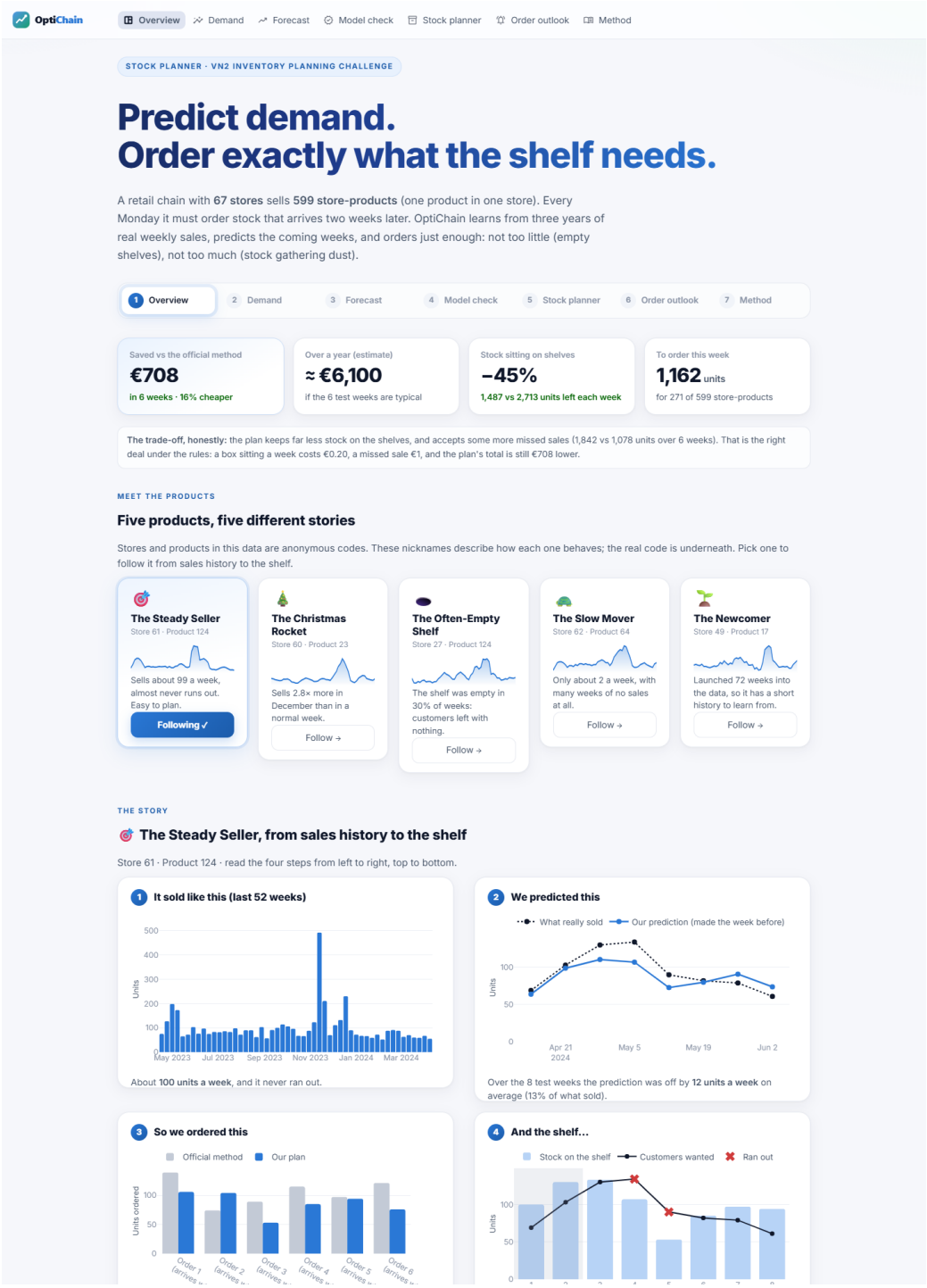


Figure 28. Dashboard: Overview page.

Demand: how does a product sell?

- Store and Product selectors (shared by all pages; hero products show their nickname).
- Cards: sales pattern, average week, share of weeks with no sale, share of weeks out of stock.
- Weekly sales: blue bars, orange bands on out-of-stock weeks, the 13-week average, grey band before launch.
- Seasonality across all stores; the pattern map (weeks between sales vs size variation) with the product highlighted.
- Availability heatmap: share of each store's launched products out of stock, month by month.



Figure 29. Dashboard: Demand page.

Forecast: which method predicts best?

- Leaderboard of the 10 methods with a choice of measure (Score, WAPE, MAE, Bias); hover for each method's idea.
- Try it: pick a product, a forecast date (30 backtest dates and 9 competition rounds) and up to three methods; see the forecasts, the 80% range and what actually sold.
- Error by sales pattern (heatmap) and LightGBM's most important inputs.
- Accuracy on the eight competition weeks.



Figure 30. Dashboard: Forecast page.

Model check: are the statistics sound?

- Whole network or one store-product (computed live).
- Verdict cards: stationary (ADF), seasonal strength, best model (AIC, with the BIC choice), residuals white noise.
- Tabs: stationarity (rolling mean and spread, ADF/KPSS table), season and trend (STL), ACF/PACF with the identified model and lags, model choice (ΔAIC bars), residuals and GARCH (tests before and after), forecast check.
- All 599: model families, share stationary, share white noise, ARCH share, and out-of-sample error whiteness per method.

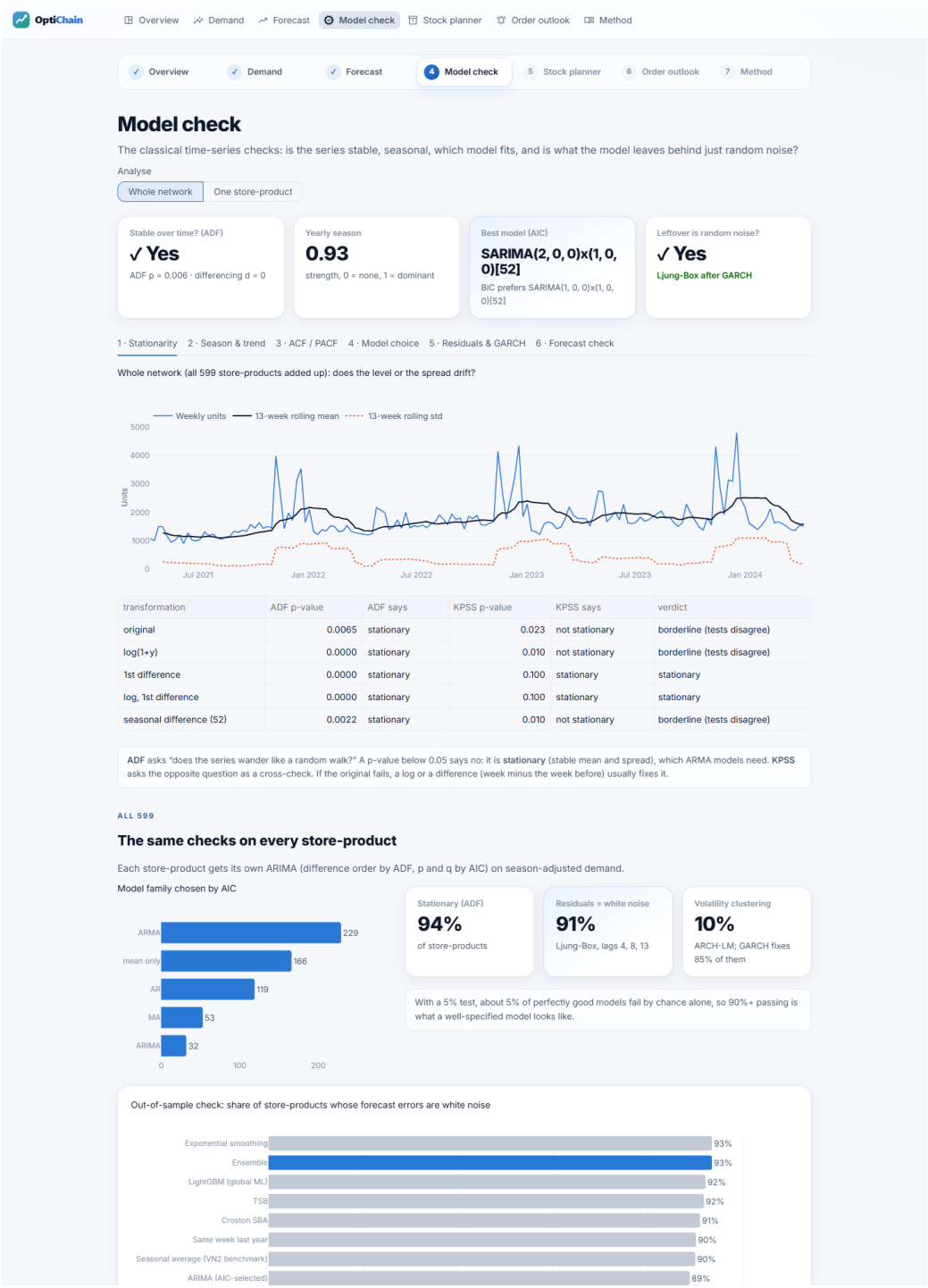
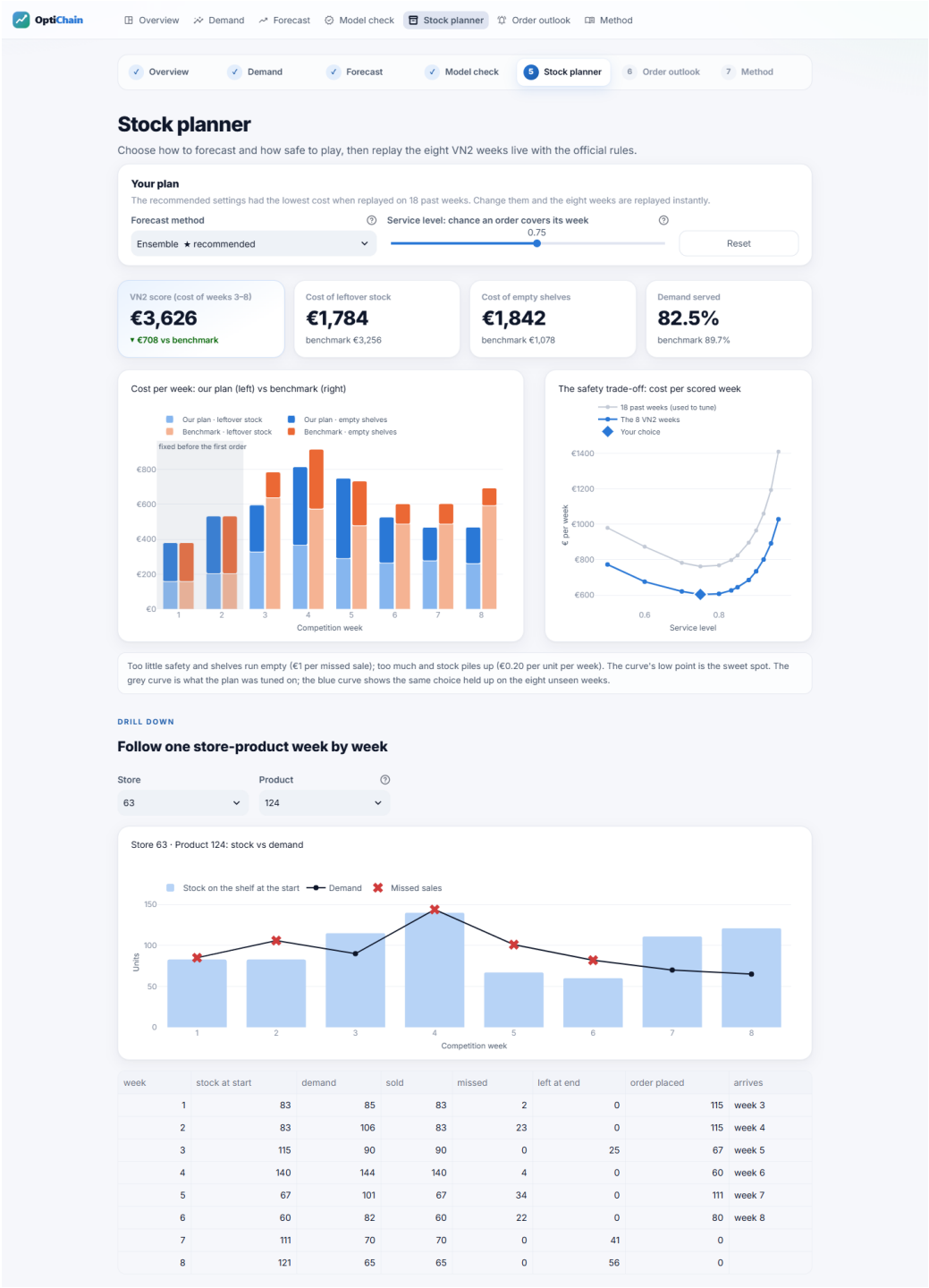


Figure 31. Dashboard: Model check page.

Stock planner: what if I change the settings?

- Choose the forecast method and the service level (slider); Reset returns to the recommended plan.
- Cards: VN2 score vs the benchmark, cost of leftover stock, cost of empty shelves, share of demand served.
- Cost per week (plan vs benchmark, stacked leftover/missed) and the safety trade-off curve (past replay and the 8 VN2 weeks, with your choice marked).
- Drill-down for one product: stock at the start of each week, demand, missed sales, the orders and when they arrive; download the six orders as CSV.



week	stock at start	demand	sold	missed	left at end	order placed	arrives
1	83	85	83	2	0	115	week 3
2	83	106	83	23	0	115	week 4
3	115	90	90	0	25	67	week 5
4	140	144	140	4	0	60	week 6
5	67	101	67	34	0	111	week 7
6	60	82	60	22	0	80	week 8
7	111	70	70	0	41	0	
8	121	65	65	0	56	0	

Figure 32. Dashboard: Stock planner page.

Order outlook: what do I order this week?

- Headline: units to order now, for how many store-products, arrival date.
- Cards: stock on the shelf and on its way, expected demand, high-risk counts.
- Chance of an empty shelf by week (low, medium, high) and the 15 stores ordering most.
- Action list of all 599 with filters (stores, pattern, minimum risk), progress bars for risk, CSV download.

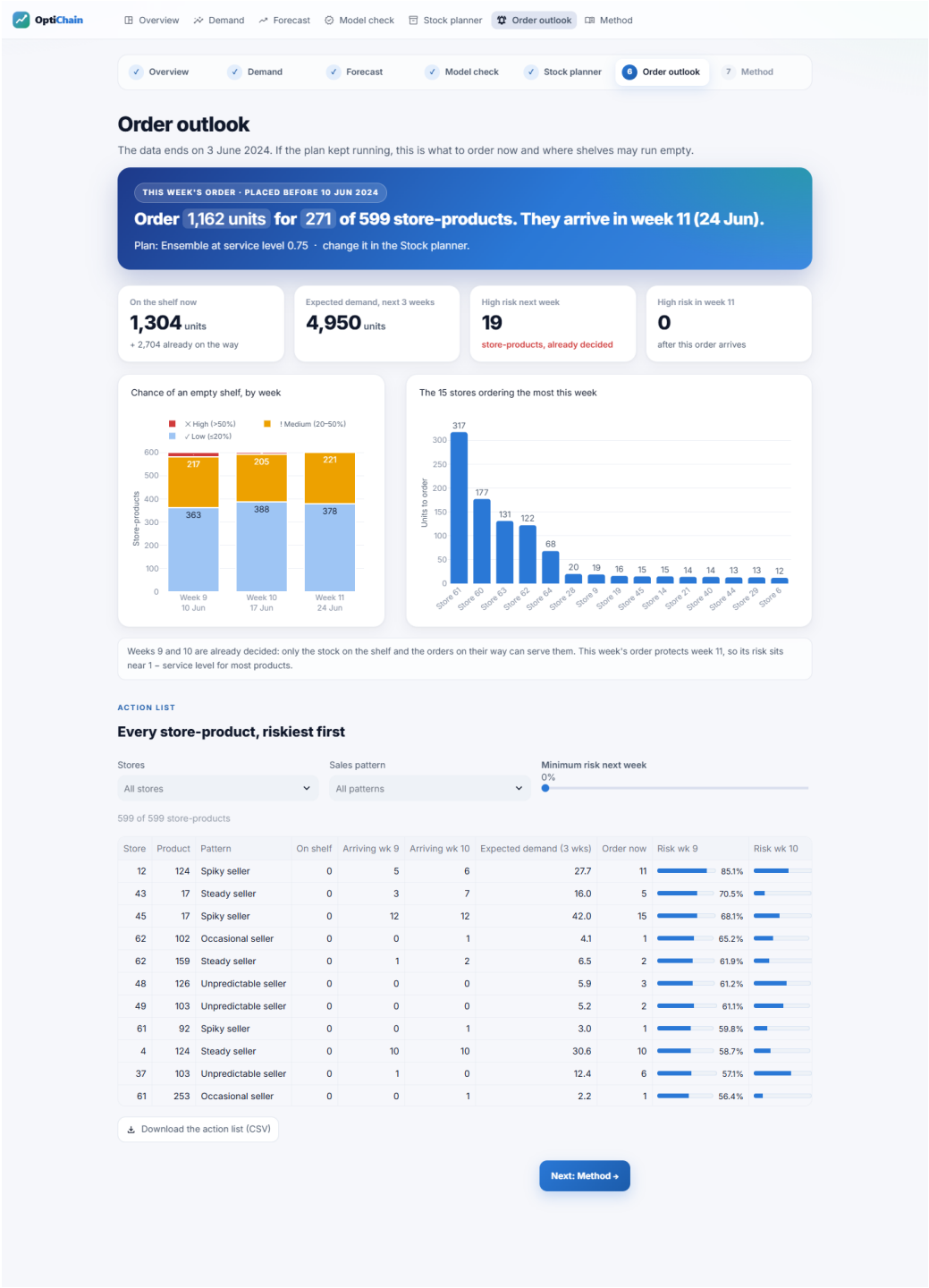


Figure 33. Dashboard: Order outlook page.

Method: how was it built?

- The six stages of the pipeline, the official rules, a glossary, the calibration table, limitations, the data citation and downloads of the main results.



Figure 34. Dashboard: Method page.

PART 14

The engineering

14.1 · Project structure

Path	Role
config.yaml	every setting in one place
src/optichain/data.py	load VN2, launch and censoring masks, Syntetos-Boylan classes, audit
src/optichain/timeseries.py	ADF/KPSS, transformations, STL, ACF/PACF, identification, AIC/BIC, Ljung-Box, Jarque-Bera, ARCH-LM, GARCH, per-product ARIMA (RowARIMA), error whiteness
src/optichain/models.py	the forecasting models, seasonal factors, LightGBM features and training
src/optichain/evaluation.py	rolling-origin backtest, metrics, spread (k) estimation
src/optichain/inventory.py	official simulator, negative-binomial sampling, newsvendor orders, replays, outlook
src/optichain/planning.py	round-by-round forecasts, policy tuning, competition scoring, the live plan
src/optichain/plots.py	the notebook's charts, one function per chart, so the notebook shows only the analysis
scripts/build_notebook.py	writes and runs the pipeline notebook
scripts/build_*.py	build the PDF documents (guide, cell-by-cell, this walkthrough)
app/	the Streamlit dashboard: Home.py, views/ (one file per page), ui.py (design system), data.py (cached loaders)
artifacts/	results written by the notebook and read by the dashboard
tests/	23 automatic tests
archive/m5/	the earlier Walmart M5 version

14.2 · Settings (config.yaml)

Setting	Value	Meaning
holding_cost / shortage_cost	0.2 / 1.0 €	official costs
lead_time_weeks	2	order k arrives at the start of week k + 2
forecast.horizon	3	weeks forecast in the backtest
forecast.backtest_origins	30	rolling forecast dates
forecast.dispersion_origins	12	first dates, used only for the spread k
forecast.lgbm_retrain_every / arima_refit_every	4 / 8	weeks between refits in the backtest
forecast.ensemble_of	LightGBM, smoothing, benchmark	ensemble members
policy.samples	1,000	Monte Carlo demand paths per store-product
policy.service_grid	0.50 ... 0.96 (12 values)	service levels tried in the tuning
policy.replay_warmup	3	replay weeks not scored
seed	42	same numbers on every run

14.3 · Tests

23 automatic tests (python -m pytest) guard the logic: censoring masks and pattern classes; every model returns finite non-negative forecasts and ignores censored weeks; Croston on a known intermittent series; the official weekly arithmetic; **the simulator reproduces €913.8 and €4,334.0**; the benchmark re-implementation places the official first order; negative-binomial sampling has the right mean and variance; newsvendor orders grow with the service level; ADF separates noise from a random walk; AIC recovers a known AR(1) with white-noise residuals; GARCH removes a simulated ARCH effect; every dashboard page renders; the planner reacts and keeps the choice; the Model check works for one product; following a hero product updates the story.

14.4 · Reproducibility and run time

The whole pipeline runs from one command (python scripts/build_notebook.py --run) in about 20-25 minutes on a 12-core laptop: the backtest about 6 minutes, the competition rounds about 6, the policy tuning about 3, the 599 ARIMA fits about 1 (in parallel). A fixed seed makes every run identical. The VN2 files ship with the project and are downloaded automatically if missing.

PART 15

Limitations and next steps

15.1 · Limitations

- **Hindsight.** The design was informed by the published competition results.
- **Censored demand in the simulation.** Demand is the recorded sales (as in the competition), so weeks that ran out in reality understate true demand.
- **Short test.** Eight weeks is a short window; a few large store-products can move the score.
- **Anonymous data.** What the products are, and their prices and margins, is unknown; costs are uniform.
- **Simple spread model.** Variance = k × mean is simple; the risk ranges are slightly generous.
- **Myopic policy.** Each order looks only at its arrival week, not at the weeks after it.

15.2 · Next steps

- Quantile or distributional LightGBM, or conformal intervals, for better-calibrated risk.
- Service levels per demand pattern or per product group.
- A multi-week policy (dynamic programming or reinforcement learning, as the 2nd place did).
- Estimating true demand in censored weeks instead of only skipping them.
- A permanent deployment (Streamlit Community Cloud) with weekly data updates.

Data: VN2 Inventory Planning Challenge (Vandeput, 2025), CC BY 4.0

PART A

Glossary

Term	Meaning
ACF / PACF	correlation of a series with itself k weeks earlier; PACF keeps only the direct link
ADF test	Augmented Dickey-Fuller; H0 = unit root (not stationary)
AIC / BIC	information criteria: fit minus a complexity penalty; lower is better
ARCH effect	volatility clustering: the size of errors changes over time in runs
AR, MA, ARMA, ARIMA, SARIMA	autoregressive, moving-average, both, with differencing, with a seasonal part
Backtest	forecasting past weeks with only the data known at the time
Bias	total over-forecast ÷ total demand
Censored demand	sales lower than demand because the shelf ran empty
Clean demand	sales with out-of-stock and pre-launch weeks removed
Critical ratio	p ÷ (p + h) = 0.833: the textbook service level
Croston SBA / TSB	forecasting methods for intermittent demand
Ensemble	the average of several models' forecasts
GARCH(1,1)	a model of changing variance: $\sigma^2 = \omega + \alpha \cdot \varepsilon^2 + \beta \cdot \sigma^2$
Holding cost	€0.20 per unit left on the shelf at the end of a week
Jarque-Bera	test of normality from skewness and kurtosis
Kalman filter	the algorithm that estimates state-space (ARIMA) models and skips missing weeks
KPSS test	H0 = stationary (the mirror of ADF)
Lead time	time between ordering and delivery: 2 weeks
LightGBM	a fast gradient-boosted decision-tree library
Ljung-Box	test for leftover autocorrelation (white noise if p > 0.05)
Lost sale / shortage cost	demand the empty shelf could not serve; €1 per unit
MAE / WAPE / Score	error measures; Score = WAPE + |Bias|
Monte Carlo	estimating by simulating many random scenarios
Negative binomial	a count distribution with variance above the mean
Newsvendor	the classic single-period ordering problem with an optimal quantile
NaN	a missing value
Order-up-to	order enough to bring stock + in transit up to N weeks of forecast
p-value	chance of the observed result if H0 were true; below 0.05 rejects H0
Service level	chance that an order covers demand in its arrival week
Stationary	mean and variance do not drift over time

Data: VN2 Inventory Planning Challenge (Vandeput, 2025), CC BY 4.0

Term	Meaning
STL	Seasonal-Trend decomposition using LOESS
Store-product	one product in one store; 599 in total
Tweedie loss	a loss function for non-negative data with many zeros
Week 0	8 April 2024, the last history week before the competition
White noise	a series with no pattern left: uncorrelated, constant variance

Data: VN2 Inventory Planning Challenge (Vandeput, 2025), CC BY 4.0

PART B

Questions and answers

Question	Answer
Why VN2 and not Walmart?	It is recent and real, with real stock levels, costs, rules and an official score.
What is a store-product?	One product in one store. 67 stores and 297 products give 599 existing pairs; each is planned separately.
Why set out-of-stock weeks aside?	All of them recorded zero sales: those zeros measure an empty shelf, not customer demand.
Why both ADF and KPSS?	They test opposite hypotheses; agreement makes the conclusion safer.
Why no differencing?	ADF already rejects a unit root on the levels (d = 0), and AIC confirmed it.
Why SARIMA for the total?	AR signature in ACF/PACF, strong yearly season (strength 0.93), and the lowest AIC.
Why GARCH?	ARCH-LM found volatility clustering; after GARCH the residuals are white noise.
What does Ljung-Box prove?	That no pattern is left in the residuals or errors. Accuracy is proven separately on unseen weeks.
Why is ARIMA not the final model?	Tested fairly, LightGBM and the ensemble were more accurate and cheaper; ARIMA still beat the official method.
Why LightGBM?	One model learns from all 599 store-products at once, so slow sellers borrow patterns from similar ones.
Why a negative binomial?	Weekly sales are small whole numbers with many zeros and extra spread; a bell curve does not fit.
Why 0.75 and not 0.833?	Tuned by simulating real costs on past weeks; the spread is slightly generous and leftovers sell later.
Why does the plan miss more sales than the benchmark?	Holding 45% less stock saves more than the extra missed sales cost under the €1 vs €0.20 rules.
Is beating 1st place a fair claim?	No: the project was built after the competition. The fair claim is 16% cheaper than the official method.
Why does the outlook say 24 June 2024?	The data ends on 3 June; the order placed on 10 June arrives two weeks later.
Is the simulator trustworthy?	It reproduces the official €4,334.0 to the cent with identical orders.

PART C

The notebook, cell by cell

The following pages explain every one of the 108 cells of the pipeline notebook in order: what each cell does, what it shows and why it matters, with the start of its code and its real output.

NOTEBOOK GUIDE

The OptiChain notebook, cell by cell

Every one of the 105 cells of *notebooks/OptiChain_VN2_Pipeline.ipynb* explained in simple words: 52 code cells and 53 text cells, with a short look at each cell's code and its real output from the last run.

How to read this guide

Box	What it shows
What it does	the cell's job, in plain words
What you see	the result it prints or draws, as in the last run
Why it matters	why this step is in the project
Code (start)	the first lines of the cell's code, for reference
Output	the real printed output; charts are shown as small pictures
Text cell	a short summary of the explanation written in the notebook

Short vocabulary. *Store-product* = one product in one store (599 in total). *Week 0* = 8 April 2024, the last week of history. *Clean demand* = sales with out-of-stock and pre-launch weeks removed. *NaN* = an empty (missing) value. *p-value* = the chance of seeing the result if the test's starting assumption were true; below 0.05 means "reject it".

Cell 0 · TEXT CELL

The title page of the notebook: the business problem, the data, and a table of all 20 steps.

0 · Setup

Cell 1 · TEXT CELL

Heading: step 0, setup.

Cell 2 · CODE CELL

What it does. Loads the tools (pandas for tables, numpy for maths) and the project's own modules, each with a short comment: D = data, E = evaluation, I = inventory, M = models, P = planning, plots = charts, TS = time series. It reads the settings from config.yaml, creates the artifacts folder, fixes the random seed, hides statsmodels' messages and sets one chart style.
What you see. Nothing is printed.
Why it matters. Fixing the seed (42) means every run gives exactly the same numbers. Hiding the warnings keeps the output readable.

Code (start)

```
import json
import warnings

import numpy as np
import pandas as pd

from optichain import data as D            # loading and cleaning the VN2 files
… (14 more lines)
```

1 · Data collection

Cell 3 · TEXT CELL

Heading: step 1, data collection. A table lists the competition files and what each one contains.

Cell 4 · CODE CELL

What it does. Checks that all VN2 files are in data/raw/vn2. Any missing file is downloaded from the organiser's GitHub page, and reports what it downloaded.
What you see. "Downloaded: nothing, every file is already here".
Why it matters. The notebook works on any computer: if the data is missing, it fetches it itself.

Code (start)

```
fetched = D.download_vn2(cfg.paths.raw_dir, cfg.paths.source_url)   # downloads only the files that are missing
print("Downloaded:", fetched or "nothing, every file is already here")
```

Output

```
Downloaded: nothing, every file is already here
```

Cell 5 · CODE CELL

What it does. Reads all the files into one object called panel: sales, in-stock flags, product groups and the starting stock. It also makes hist, the same data as fast number arrays for the models, and t0, the position of week 0 (8 April 2024).
What you see. 599 store-products × 165 weeks, and the first 5 rows × 8 weeks of sales.
Why it matters. Each row is one product in one store; each column is one week. Everything later works on this table.

Code (start)

```
panel = D.load_panel(cfg.paths.raw_dir, cfg.competition.week_0)    # sales, in-stock flags, groups, starting st
ock
hist = M.History.from_panel(panel)                                  # the same data as arrays, for the models
t0 = panel.t0                                                       # column of week 0 (8 April 2024)
print(f"{panel.sales.shape[0]} store-products x {panel.sales.shape[1]} weeks")
panel.sales.iloc[:5, :8]
```

Output

```
599 store-products x 165 weeks
```

Cell 6 · TEXT CELL

Text: explains the shape of the data (rows are store-products, columns are weeks).

Cell 7 · CODE CELL

What it does. Shows the first rows of three more tables: the in-stock flags (True/False), the product and store groups, and the stock on the shelf and on its way at the start.
What you see. Three small tables.
Why it matters. To see what each file looks like before using it.

Code (start)

```
display(panel.in_stock.iloc[:5, :8])    # True = stock was left at the end of the week
display(panel.master.head())            # product and store groups
panel.initial.head()                    # stock on the shelf and on its way at the start
```

Output: 3 table(s), shown in the notebook.

Cell 8 · TEXT CELL

Heading: step 1.2, the data-quality audit.

Cell 9 · CODE CELL

What it does. Runs a quality check and puts the results in one table: number of stores, products and weeks, missing values, negative values, decimals, share of zero weeks, share of out-of-stock weeks.
What you see. A table of facts: 0 missing, 0 negative, 0 decimals, 43% zero weeks, about 10% out-of-stock weeks.
Why it matters. If the data were broken, every later result would be wrong. Checking first is basic good practice.

Code (start)

```
audit = D.audit(panel)
audit
```

Output: 1 table(s), shown in the notebook.

Cell 10 · CODE CELL

What it does. Opens all 9 weekly sales files one by one and checks that each repeats the earlier weeks unchanged, keeps the same 599 rows in the same order, and adds exactly one new week.
What you see. "All 9 sales files are consistent".
Why it matters. Proves the files agree with each other, so using the last file (which holds every week) is safe.

Code (start)

```
print(D.check_weekly_files(cfg.paths.raw_dir, panel))
```

Output

```
All 9 sales files are consistent: same rows, same history, one extra week each.
```

Cell 11 · TEXT CELL

Text: reads the audit. Data is complete; 43% of weeks sell nothing; about 1 week in 10 is out of stock.

2 · Exploratory data analysis

Cell 12 · TEXT CELL

Heading: step 2, exploratory data analysis (EDA).

Cell 13 · CODE CELL

What it does. Turns the wide table (one column per week) into a long table (one row per store-product-week) with the product groups attached. Also defines hist_cols, which marks the 157 history weeks.
What you see. The first rows of the long table (98,835 rows in total).
Why it matters. A long table is easier for charts and group totals. hist_cols makes sure the 8 test weeks stay hidden.

Code (start)

```
long = D.to_long(panel)                          # one row per store-product-week
hist_cols = panel.sales.columns <= panel.week0   # the 157 history weeks; the 8 test weeks stay unseen
long.head()
```

Output: 1 table(s), shown in the notebook.

Cell 14 · TEXT CELL

Heading: 2.1, line graph of the whole network over time.

Cell 15 · CODE CELL

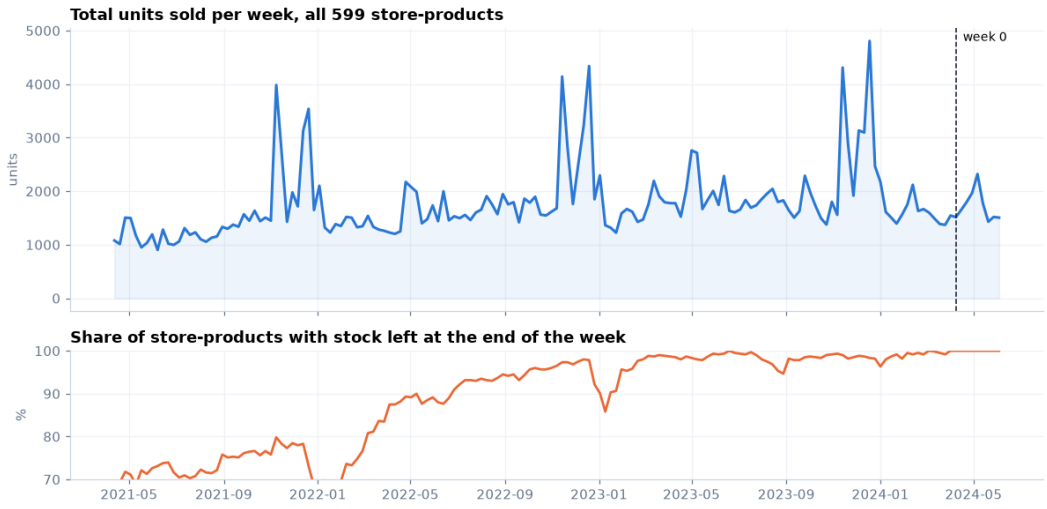
What it does. Adds up the sales of all 599 store-products for each week, and the share of store-products that had stock left. Draws both as line graphs.
What you see. Top: total units per week with yearly peaks. Bottom: the in-stock share, which dips in some periods. Mean about 1,761 units a week.
Why it matters. Shows the big picture: a stable level, a strong Christmas peak, and periods with many empty shelves.

Code (start)

```
total = panel.sales.sum(axis=0)                    # units sold by all store-products, per week
instock_share = panel.in_stock.mean(axis=0)        # share of store-products with stock left, per week
plots.network_over_time(total, instock_share, panel.week0)
print(f"Average week: {total.mean():,.0f} units")
```

Output

```
Average week: 1,761 units
```



Cell 16 · TEXT CELL

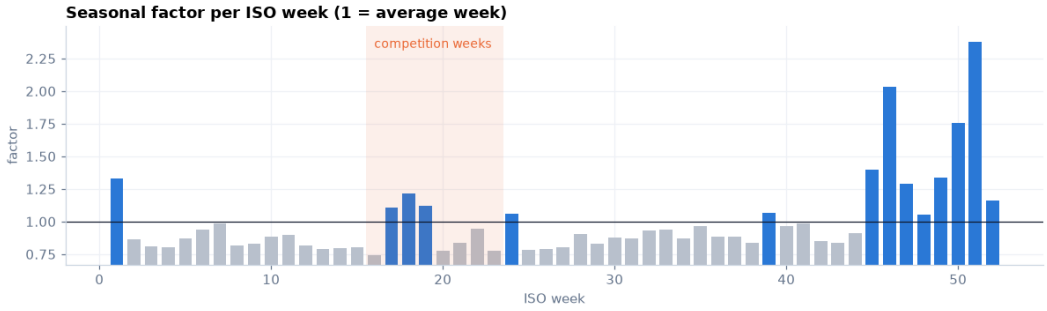
Text: sales are stable with a yearly rhythm; availability is not, so the in-stock flag matters.

Cell 17 · CODE CELL

What it does. Computes the seasonal factor of each week of the year: the average sales in that week ÷ the average week. This is exactly how the official benchmark measures seasonality.
What you see. A bar chart with 52 bars; weeks above 1 sell more than average; the test weeks (16-23) are shaded.
Why it matters. Every seasonal model later uses these factors.

Code (start)

```
season = M.season_factors(hist.upto(t0))           # average sales of each ISO week / average week
plots.seasonal_factors(season)
```



Cell 18 · TEXT CELL

Heading: 2.3, histograms (how much does one store-product sell?).

Cell 19 · CODE CELL

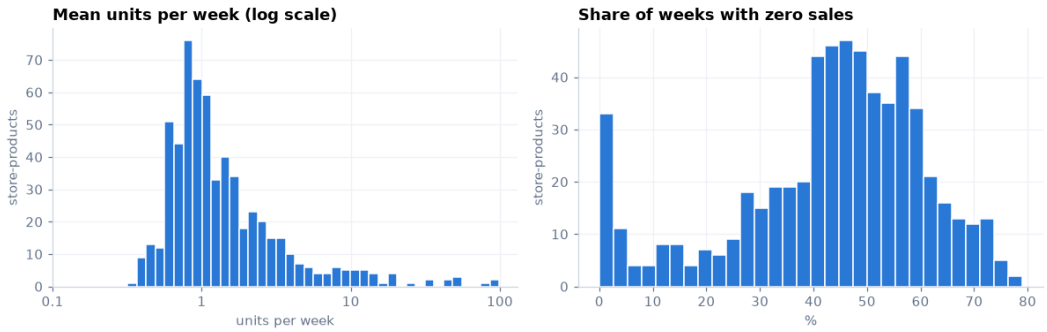
What it does. Computes, for each store-product, its average weekly sales and its share of zero weeks, and draws two histograms.
What you see. Most store-products sell about 1 unit a week; 75% sell under 2 a week.
Why it matters. Shows that demand is small and slow, so the models must handle many zeros.

Code (start)

```
sales_h = panel.sales.loc[:, hist_cols]            # history weeks only
mean_week = sales_h.mean(axis=1)                   # average units per week, per store-product
zero_share = (sales_h == 0).mean(axis=1)           # share of weeks with no sale
plots.sales_histograms(mean_week, zero_share)
print(f"Median store-product: {mean_week.median():.1f} units/week; {(mean_week < 2).mean():.0%} sell under 2 a
week")
```

Output

```
Median store-product: 1.1 units/week; 75% sell under 2 a week
```



Cell 20 · TEXT CELL

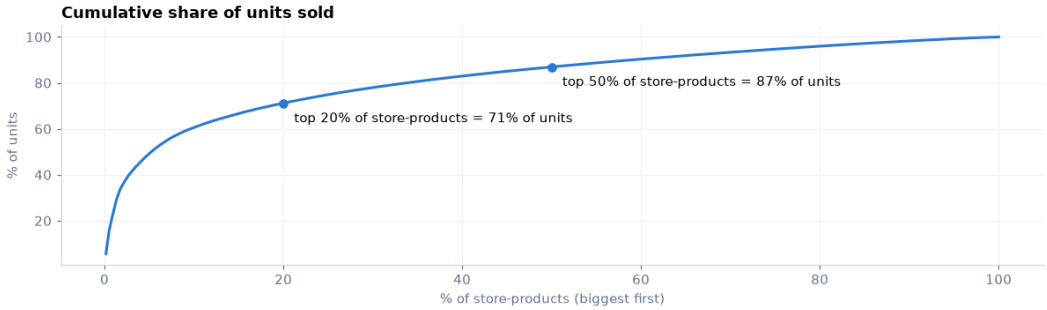
Text: demand is very skewed; errors measured in units are dominated by the big sellers. Heading 2.4: Pareto.

Cell 21 · CODE CELL

What it does. Sorts store-products from biggest to smallest seller and draws the running total of their share of sales (a Pareto curve).
What you see. A curve with two marked points, for example "top 20% of rows = X% of units".
Why it matters. Shows where the volume is: a small group of products makes most of the sales.

Code (start)

```
volume = sales_h.sum(axis=1).sort_values(ascending=False)    # total units per store-product, biggest first
plots.pareto(volume.cumsum() / volume.sum())
```



Cell 22 · TEXT CELL

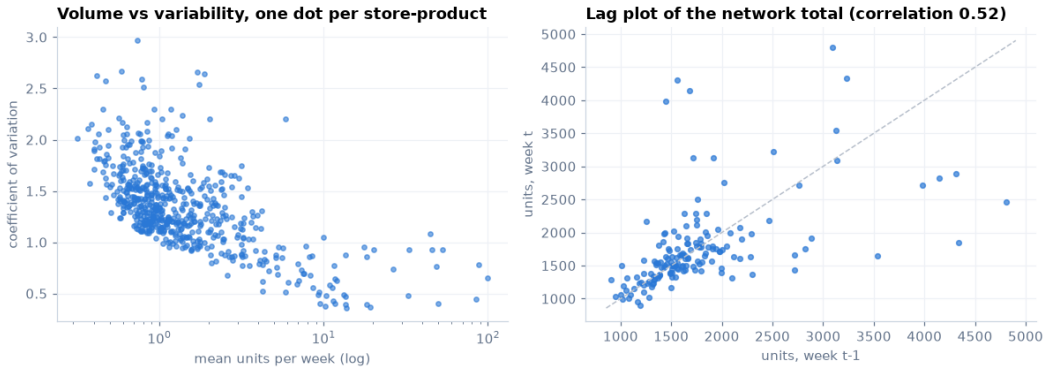
Heading: 2.5, scatter plots, and what they show.

Cell 23 · CODE CELL

What it does. Left: each store-product as a dot, average sales against variability (coefficient of variation). Right: the network total, this week against last week (a lag plot).
What you see. Slow sellers are the most variable. The lag-plot dots lie near a diagonal line (high correlation).
Why it matters. The lag plot shows that this week depends on last week, which is what AR (autoregressive) models use.

Code (start)

```
total_h = total.loc[:panel.week0].to_numpy()                # network total, history weeks
cv = sales_h.std(axis=1) / mean_week                        # coefficient of variation per store-product
plots.scatter_plots(mean_week, cv, total_h)
```



Cell 24 · TEXT CELL

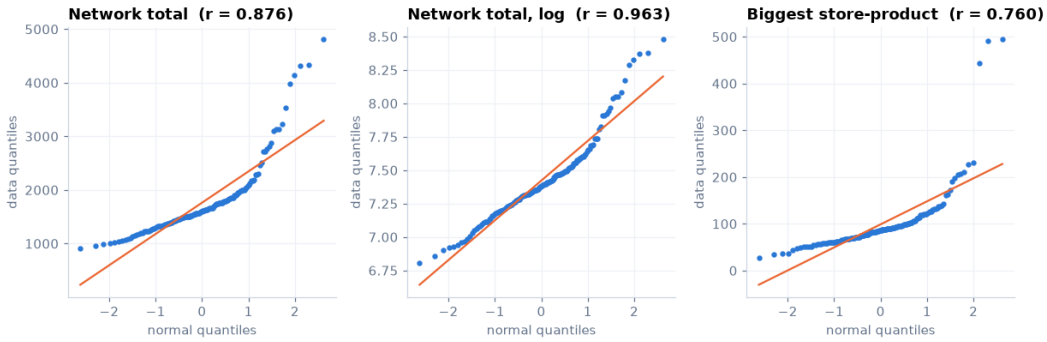
Text: reads the scatter plots. Heading 2.6: QQ plots, and what a QQ plot is.

Cell 25 · CODE CELL

What it does. Draws QQ plots for the network total, its log, and the biggest single store-product. Each plot compares the data with a perfect normal (bell-curve) distribution.
What you see. Points follow the line in the middle but bend away at the top (heavy tail).
Why it matters. Sales are not normally distributed. That decides later choices, such as using a count distribution for risk.

Code (start)

```
biggest = int(sales_h.sum(axis=1).to_numpy().argmax())       # the store-product with the most sales
plots.qq_plots({"Network total": total_h, "Network total, log": np.log(total_h),
                "Biggest store-product": sales_h.iloc[biggest].to_numpy()})
```



Cell 26 · TEXT CELL

Text: the total is close to normal in the middle with a heavy upper tail; single products are far from normal. Heading 2.7.

Cell 27 · CODE CELL

What it does. Computes descriptive statistics (mean, standard deviation, quartiles, CV, skewness, kurtosis, share of zero weeks, Jarque-Bera p-value) for the total, the biggest seller and the median product, then summarises the same statistics across all 599.
What you see. Two tables of numbers.
Why it matters. Puts numbers on what the charts showed: right-skewed, heavy-tailed, many zeros, not normal.

Code (start)

```
desc = TS.describe({"Network total": total_h, "Biggest store-product": sales_h.iloc[biggest].to_numpy(),
                    "Median of all store-products": sales_h.median(axis=0).to_numpy()})
display(desc.T.round(2))
per_row = TS.describe({i: sales_h.iloc[i].to_numpy() for i in range(hist.n)})   # the same for each store-produ
ct
per_row[["mean", "CV", "skewness", "excess kurtosis", "zero weeks %"]].astype(float).describe().round(2)
```

Output: 2 table(s), shown in the notebook.

Cell 28 · TEXT CELL

Text: reads the statistics. Heading 2.8: stockouts and product groups.

Cell 29 · CODE CELL

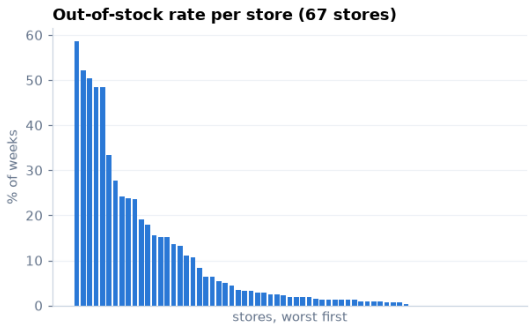
What it does. Computes the out-of-stock rate of each of the 67 stores and total sales per department group, and charts both.
What you see. Stores range from 0% to 59% of weeks out of stock; one chart of units per department group.
Why it matters. Shows that empty shelves are a real and uneven problem across stores.

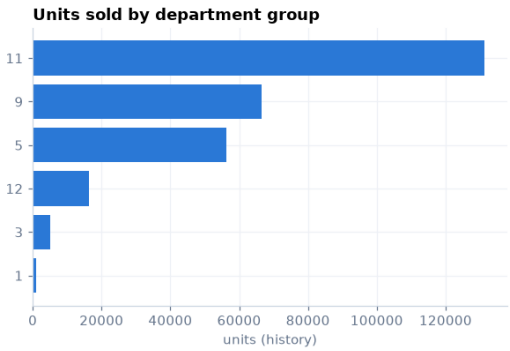
Code (start)

```
oos = 1 - panel.in_stock.loc[:, hist_cols].mean(axis=1)                    # out-of-stock rate per store-pro
duct
store_oos = oos.groupby(level="Store").mean().sort_values(ascending=False)     # ... per store
dept_sales = long[long.phase == "history"].groupby("DepartmentGroup")["sales"].sum().sort_values()
plots.stockouts_and_groups(store_oos, dept_sales)
print(f"Out-of-stock rate per store: from {store_oos.min():.0%} to {store_oos.max():.0%}")
```

Output

```
Out-of-stock rate per store: from 0% to 59%
```





Cell 30 · TEXT CELL

Text: what the EDA teaches us and how it shapes the models (slow demand, shared season, censoring, shared patterns).

3 · Dealing with missing values

Cell 31 · TEXT CELL

Heading: step 3, missing values. There are no empty cells, but two kinds of weeks are hidden missing values.

Cell 32 · TEXT CELL

Heading 3.1 and text: zeros before a product's first sale mean it was not on sale yet, not that nobody wanted it.

Cell 33 · CODE CELL

What it does. Finds each store-product's first week with a sale (its launch) and marks every earlier week. Draws a histogram of launch weeks.
What you see. 38% sell from the first week; 371 store-products were launched later.
Why it matters. Those early zeros would wrongly drag forecasts down, so they are treated as missing.

Code (start)

```
launched = D.launch_mask(panel.sales)                    # True from the first week with a sale
first_sale = launched.to_numpy().argmax(axis=1)          # week number of the first sale
print(f"Selling from the first week: {(first_sale == 0).mean():.0%} · launched later: {(first_sale > 0).sum()}"
)
plots.launch_histogram(first_sale)
```

Output

```
Selling from the first week: 38% · launched later: 371
```



Cell 34 · TEXT CELL

Heading 3.2 and text: out-of-stock weeks are censored (sales understate demand), so they are set aside.

Cell 35 · CODE CELL

What it does. Builds demand: the sales table with out-of-stock weeks and pre-launch weeks replaced by NaN (empty). Then measures how much is set aside and what out-of-stock weeks look like.
What you see. 4.3% of weeks set aside; 100% of out-of-stock weeks have zero sales.
Why it matters. This is the key cleaning step. An empty shelf means nothing could sell, so that zero says nothing about demand.

Code (start)

```
demand = D.clean_demand(panel.sales, panel.in_stock)      # out-of-stock and pre-launch weeks become NaN (missi
ng)

in_stock_h, launched_h = panel.in_stock.loc[:, hist_cols], launched.loc[:, hist_cols]
censored = ~in_stock_h & launched_h                       # out-of-stock weeks after launch
print(f"Weeks set aside as censored: {censored.to_numpy().mean():.1%}")
print(f"Out-of-stock weeks with zero sales: {(sales_h[censored].stack().dropna() == 0).mean():.0%}")
```

Output

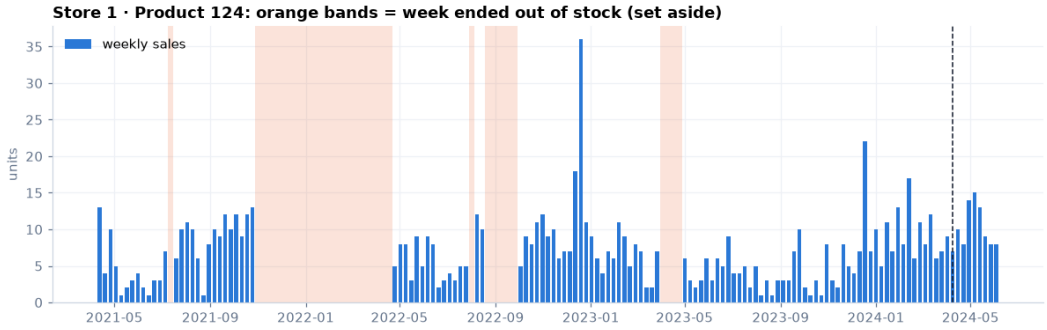
```
Weeks set aside as censored: 4.3%
Out-of-stock weeks with zero sales: 100%
```

Cell 36 · CODE CELL

What it does. Picks a busy store-product (more than 3 units a week) that often ran out (over 20% of weeks), and charts its sales with orange bands on out-of-stock weeks.
What you see. A bar chart with orange bands.
Why it matters. Makes the problem visible: the bands fall exactly on the weeks with zero sales.

Code (start)

```
oos_rate = (~in_stock_h).mean(axis=1)
example = oos_rate[(oos_rate > 0.2) & (mean_week > 3)].index[0]    # a busy store-product that often ran out
plots.stockout_example(panel.sales.loc[example], panel.in_stock.loc[example], panel.week0,
                       f"Store {example[0]} · Product {example[1]}")
```



Cell 37 · TEXT CELL

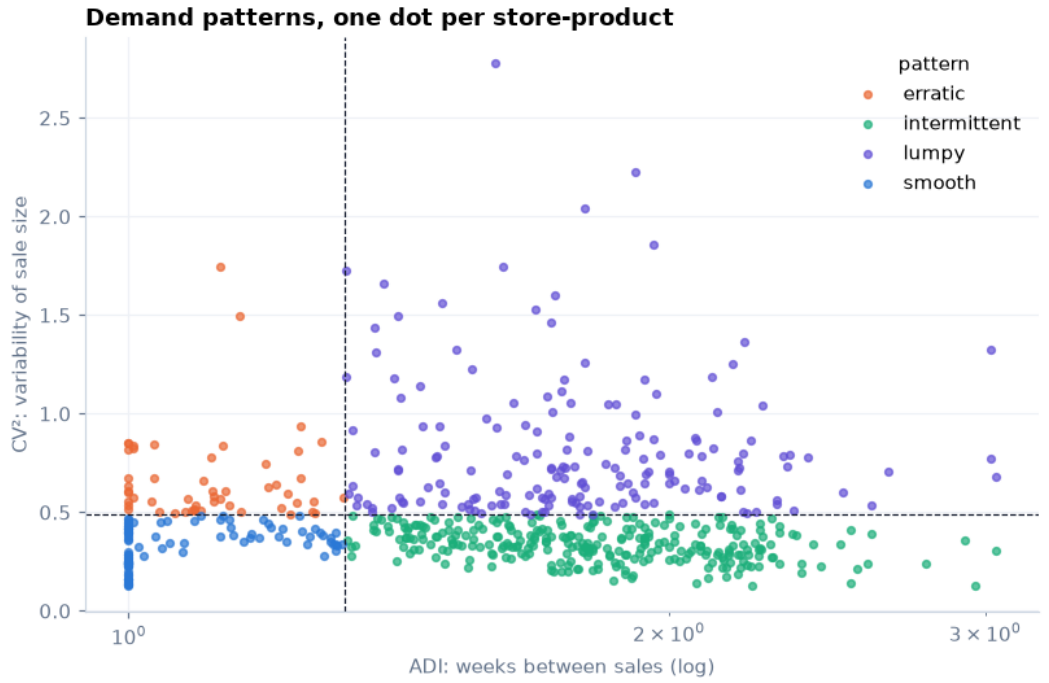
Text: every flagged week had zero sales. Then 3.3, a table of how each method treats a missing week, and 3.4, demand patterns (ADI and CV²).

Cell 38 · CODE CELL

What it does. Classifies every store-product with two numbers: ADI (average weeks between sales) and CV² (how much sale sizes vary). The standard cut-offs give 4 patterns: smooth, erratic, intermittent, lumpy.
What you see. A count table and a scatter plot with four coloured groups.
Why it matters. Each pattern suits different methods; Croston and TSB were designed for the intermittent and lumpy ones.

Code (start)

```
classes = D.classify_demand(demand.loc[:, hist_cols])     # ADI, CV² and pattern per store-product
display(classes["pattern"].value_counts())
plots.pattern_map(classes, D.ADI_CUT, D.CV2_CUT)
```



Cell 39 · TEXT CELL

Heading 3.5 and a table: how the 165 weeks are split (training, backtest, tuning, final test). No peeking at the future.

Cell 40 · CODE CELL

What it does. Defines the 30 backtest dates (origins): the first 12 for measuring forecast spread, the last 18 for accuracy and tuning. Draws a timeline of how the weeks are used.
What you see. A coloured bar timeline.
Why it matters. Keeps the test honest: the 8 competition weeks are never used to tune anything.

Code (start)

```
origins = list(range(t0 - 3 - cfg.forecast.backtest_origins + 1, t0 - 2))   # 30 past forecast dates
disp_origins = origins[:cfg.forecast.dispersion_origins]                    # first 12: error spread only
tune_origins = origins[cfg.forecast.dispersion_origins:]                    # last 18: accuracy and policy tuni
ng
plots.time_split(panel.sales.shape[1], t0, disp_origins, tune_origins, panel.dates)
```



4 · Stationarity: the ADF test

Cell 41 · TEXT CELL

Heading: step 4, stationarity. Explains the ADF test (H0: not stationary, p < 0.05 means stationary) and KPSS (the opposite), and the two series studied.

Cell 42 · CODE CELL

What it does. Picks the representative store-product (high volume, on sale from week 1, rarely out of stock). Builds y_net (the network total) and y_row (that product's clean demand, with gaps filled for the tests). Runs ADF and KPSS on both.
What you see. Store 61 · Product 124 (about 99 a week). A table: ADF says stationary for both.
Why it matters. ARIMA-type models need a stationary series, so this is checked first.

Code (start)

```
# The representative store-product: biggest seller among those sold from week 1, rarely out of stock, steady pa
ttern
steady = (first_sale == 0) & (oos_rate.to_numpy() < .05) & (classes["pattern"].to_numpy() == "smooth")
ts_row = int(np.flatnonzero(steady)[sales_h[steady].sum(axis=1).to_numpy().argmax()])
ts_name = f"Store {panel.index[ts_row][0]} · Product {panel.index[ts_row][1]}"

y_net = total_h                                          # series 1: the network total
y_row = demand.iloc[ts_row, :t0 + 1].to_numpy()          # series 2: clean demand of one store-product
… (4 more lines)
```

Output

```
Representative store-product: Store 61 · Product 124
```

Cell 43 · CODE CELL

What it does. Runs the ADF test on every one of the 599 store-products (clean weeks, gaps filled).
What you see. 94% stationary, 6% not.
Why it matters. Shows the result holds for almost every product, not just the two examples.

Code (start)

```
adf_all = TS.adf_all_rows(demand.loc[:, hist_cols])        # ADF p-value of every store-product
print(f"ADF on {len(adf_all)} store-products: {np.mean(adf_all < .05):.0%} stationary")
```

Output

```
ADF on 599 store-products: 94% stationary
```

5 · Transformations

Cell 44 · TEXT CELL

Text: ADF and KPSS readings. Heading: step 5, transformations (log, first difference, seasonal difference).

Cell 45 · CODE CELL

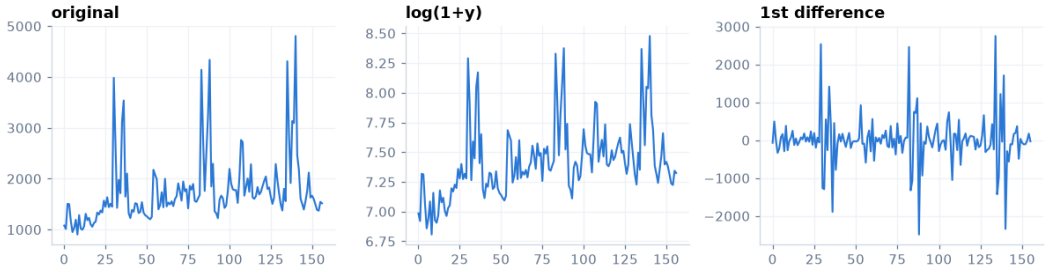
What it does. Applies each transformation to the network total, re-tests ADF and KPSS, and chooses d (how many times to difference) with the ADF rule. Plots three versions.
What you see. A results table and "d = 0" for both series (no differencing needed).
Why it matters. Proves the original series can be used as it is, so forecasts stay in real units.

Code (start)

```
tr = TS.transformations(y_net, 52)                          # original, log, 1st difference, seasonal difference
tr_tab = TS.stationarity(tr)
display(tr_tab[["ADF p-value", "ADF says", "KPSS p-value", "KPSS says", "verdict"]])
d_net, d_row = TS.choose_d(y_net), TS.choose_d(y_row_filled)   # difference only while ADF finds a unit root
print(f"Differencing chosen: network total d = {d_net}, {ts_name} d = {d_row}")
plots.transformations(tr)
```

Output

```
Differencing chosen: network total d = 0, Store 61 · Product 124 d = 0
```



6 · Seasonality: STL decomposition

Cell 46 · TEXT CELL

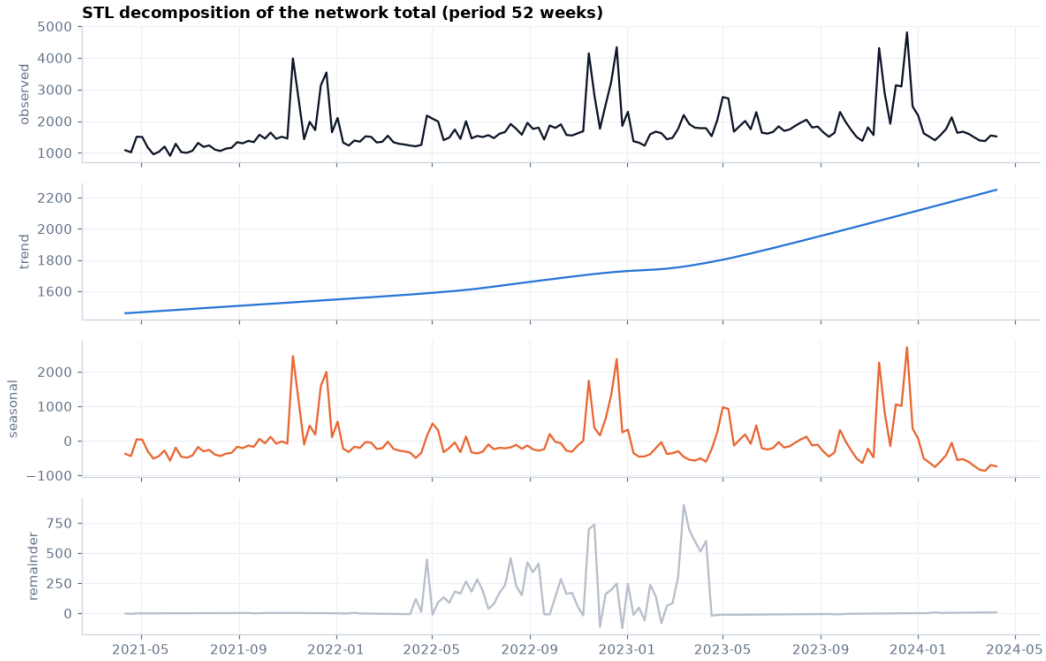
Text: why d = 0 is kept. Heading: step 6, STL decomposition and what trend and season strength mean.

Cell 47 · CODE CELL

What it does. Splits the network total into trend, seasonal and remainder with STL (52-week season), plots the four parts, and computes trend and seasonal strength for both series.
What you see. A four-panel chart; seasonal strength 0.93 for the total.
Why it matters. The data is very seasonal (Christmas), so a seasonal part is needed in the models.

Code (start)

```
dec = TS.decompose(y_net, dates_h, 52)                      # trend + seasonal + remainder
plots.decomposition(dec)
s_net = TS.strength(dec)
s_row = TS.strength(TS.decompose(y_row_filled, dates_h, 52))
pd.DataFrame({"Network total": s_net, ts_name: s_row}).T.round(2)
```



7 · ACF and PACF

Cell 48 · TEXT CELL

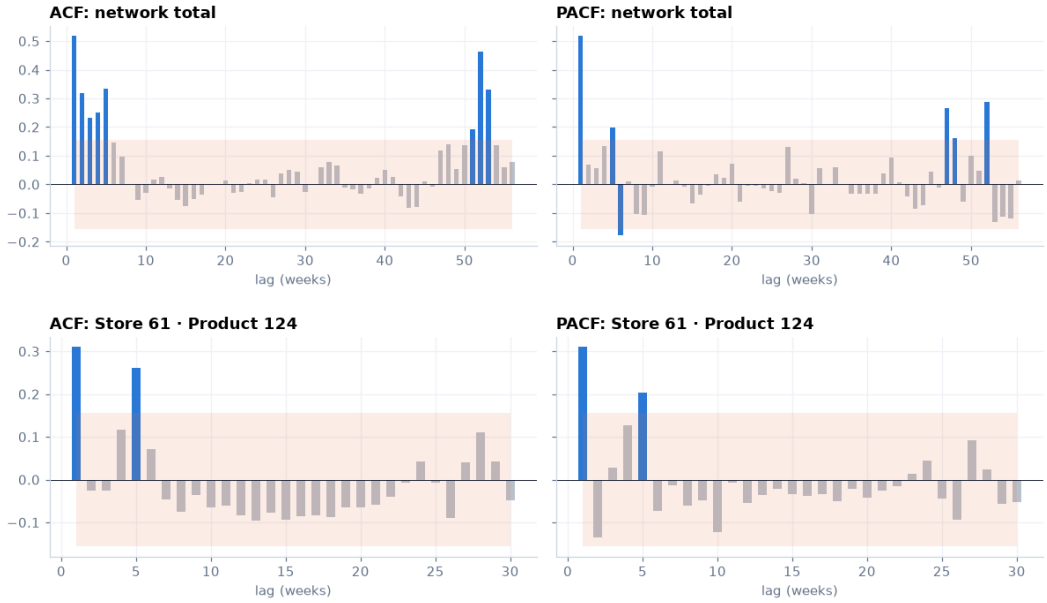
Text: reads the decomposition. Heading: step 7, what ACF and PACF are.

Cell 49 · CODE CELL

What it does. Computes the ACF and PACF of the network total (56 lags) and of the representative product (30 lags), and draws them with the 95% band. No differencing is applied because d = 0.
What you see. Two pairs of bar charts; blue bars cross the band (significant), grey bars do not.
Why it matters. These plots show which past weeks matter, which guides the choice of model.

Code (start)

```
ap_net = TS.acf_pacf(y_net, 56)                    # d = 0, so the series is used as it is
ap_row = TS.acf_pacf(y_row_filled, 30)
plots.acf_pacf(ap_net, "network total")
plots.acf_pacf(ap_row, ts_name)
```



8 · Identify the model: AR, MA, ARMA, ARIMA or SARIMA?

Cell 50 · TEXT CELL

Heading: step 8, model identification, with the textbook rules table (AR, MA, ARMA, ARIMA, SARIMA).

Cell 51 · CODE CELL

What it does. Reads the ACF/PACF automatically with those rules and suggests a model family and orders (p, d, q).
What you see. Network total: AR with a seasonal spike, so SARIMA. The product: ARMA.
Why it matters. Turns the plots into a starting model, which step 10 then checks with AIC/BIC.

Code (start)

```
id_net = TS.identify(ap_net, d_net, seasonal_lag=52)       # reads the plots with the Box-Jenkins rules
id_row = TS.identify(ap_row, d_row)
pd.DataFrame({"Network total": id_net, ts_name: id_row}).astype(str)
```

Output: 1 table(s), shown in the notebook.

9 · Identify the lags

Cell 52 · TEXT CELL

Text: why the total looks like a seasonal AR process. Heading: step 9, lags.

Cell 53 · CODE CELL

What it does. Lists the significant lags of the ACF and PACF for both series.
What you see. Network total: lags 1-5 and around 52. The product: lags 1 and 5.
Why it matters. Recent weeks and the same week last year carry the information; this also chose LightGBM's inputs.

Code (start)

```
print("Network total:", TS.significant_lags(ap_net))
print(ts_name + ":", TS.significant_lags(ap_row))
```

Output

```
Network total: {'acf': [1, 2, 3, 4, 5, 51, 52, 53], 'pacf': [1, 5, 6, 47, 48, 52]}
Store 61 · Product 124: {'acf': [1, 5], 'pacf': [1, 5]}
```

10 · AIC and BIC: choose the best order

Cell 54 · TEXT CELL

Text: what the lags mean. Heading: step 10, AIC and BIC formulas and the grid to compare.

Cell 55 · CODE CELL

What it does. Fits 27 candidate models to the network total (ARMA, ARIMA with d = 1, and SARIMA with a 52-week seasonal term) and ranks them by AIC; also shows the BIC winner.
What you see. Best by AIC: SARIMA(2,0,0)(1,0,0)[52]; best by BIC: SARIMA(1,0,0)(1,0,0)[52]. A table of the top 10.
Why it matters. Picks the best model objectively, balancing fit against complexity.

Code (start)

```
# 27 candidates: ARMA and SARIMA (seasonal AR at lag 52) with d = 0, and ARIMA with d = 1
grid_all = pd.concat([TS.aic_bic_grid(y_net, d_net, seasonal_orders=[(0, 0, 0, 0), (1, 0, 0, 52)]),
                      TS.aic_bic_grid(y_net, 1)]).sort_values("AIC").reset_index(drop=True)
grid_all["ΔAIC"] = grid_all["AIC"] - grid_all["AIC"].min()
best_net = grid_all.iloc[0]                                  # lowest AIC
best_bic = grid_all.sort_values("BIC").iloc[0]               # lowest BIC
print("Best by AIC:", best_net["model"], "· best by BIC:", best_bic["model"])
… (1 more lines)
```

Output

```
Best by AIC: SARIMA(2, 0, 0)x(1, 0, 0)[52] · best by BIC: SARIMA(1, 0, 0)x(1, 0, 0)[52]
```

Cell 56 · CODE CELL

What it does. Does the same AIC/BIC comparison for the representative product. Missing weeks stay missing; the Kalman filter skips them.
What you see. Best: MA(1) by both AIC and BIC.
Why it matters. Shows the method works for a single product too.

Code (start)

```
grid_row = TS.aic_bic_grid(y_row, d_row)                     # missing weeks stay missing: the Kalman filter skip
s them
print(ts_name, "· best by AIC:", grid_row.iloc[0]["model"])
grid_row[["model", "AIC", "BIC", "ΔAIC"]].head(6).round(1)
```

Output

```
Store 61 · Product 124 · best by AIC: MA(0, 0, 1)
```

Cell 57 · TEXT CELL

Text: the same recipe will now run automatically on all 599 store-products (season removed first).

Cell 58 · CODE CELL

What it does. Fits an ARIMA to every store-product in parallel: ADF chooses d, AIC chooses p and q, and products with fewer than 26 clean weeks get a simple mean.
What you see. Counts per family (ARMA 229, mean only 166, AR 119, MA 53, ARIMA 32) and the most common orders.
Why it matters. This per-product ARIMA becomes one of the forecasting methods in step 12.

Code (start)

```
arima_rows = TS.RowARIMA().fit(hist.upto(t0))                # one ARIMA per store-product (runs in parallel, ~1
min)
arima_tab = arima_rows.table
display(arima_tab["family"].value_counts())
arima_tab["order"].astype(str).value_counts().head(8)        # most common (p, d, q)
```

Output: 2 table(s), shown in the notebook.

11 · Residual diagnostics: is what is left white noise? ARCH and GARCH

Cell 59 · TEXT CELL

Text: reads the result. Heading: step 11, residual tests (Ljung-Box, Jarque-Bera, ARCH-LM) and GARCH.

Cell 60 · CODE CELL

What it does. Fits the winning SARIMA to the network total, prints its coefficients, and runs the three tests on its residuals (what it could not explain).
What you see. A coefficient table (all significant) and test results: Ljung-Box borderline at lag 8, ARCH effect found (p = 0.027).
Why it matters. Checks whether the model left any pattern behind.

Code (start)

```
fit_net = TS.fit_arima(y_net, best_net["order"], best_net["seasonal"])
burn = int(fit_net.loglikelihood_burn) + 1                # the first residuals have no past yet: skip them
resid_net = np.asarray(fit_net.resid)[burn:]
rt_net = TS.residual_tests(resid_net, n_params=len(fit_net.params) - 1)
print(fit_net.summary().tables[1])
pd.Series(rt_net, name=best_net["model"])
```

Output

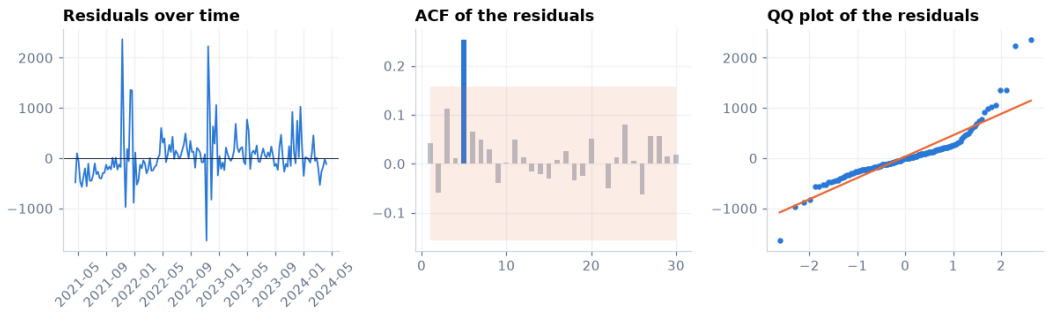
```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
intercept    280.4228     65.020      4.313      0.000     152.985     407.860
ar.L1          0.3223      0.042      7.735      0.000       0.241       0.404
ar.L2          0.1519      0.049      3.085      0.002       0.055       0.248
ar.S.L52       0.6950      0.028     24.790      0.000       0.640       0.750
sigma2      1.482e+05   9607.202     15.427      0.000    1.29e+05    1.67e+05
==============================================================================
```

Cell 61 · CODE CELL

What it does. Draws three diagnostic charts of the residuals: over time, their ACF, and a QQ plot.
What you see. Residuals hover around 0; almost all ACF bars are inside the band; the QQ plot has heavy tails.
Why it matters. A visual check that matches the tests.

Code (start)

```
plots.residual_diagnostics(dates_h[burn:], resid_net, TS.acf_pacf(resid_net, 30))
```



Cell 62 · TEXT CELL

Text: reads the charts; the ARCH effect means calm and turbulent periods alternate, so GARCH is next.

Cell 63 · CODE CELL

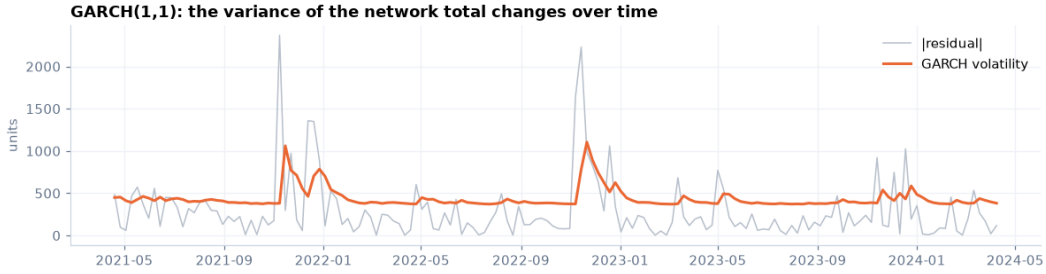
What it does. Fits GARCH(1,1) to the residuals, plots the changing volatility, and repeats the tests on the standardised residuals and their squares.
What you see. ω = 0.35, α = 0.18, β = 0.45. After GARCH, Ljung-Box passes and the ARCH effect is gone.
Why it matters. Confirms the leftover is white noise once the changing noise level is modelled.

Code (start)

```
garch_net, gscale = TS.fit_garch(resid_net)               # GARCH(1,1) on the residuals
print(garch_net.params.round(3))
std_net = np.asarray(garch_net.std_resid)                 # residual / its GARCH volatility
gtests = pd.DataFrame({"SARIMA residuals": rt_net, "after GARCH": TS.residual_tests(std_net),
                       "after GARCH, squared": TS.residual_tests(std_net ** 2)})
plots.garch_volatility(dates_h[burn:], resid_net, np.asarray(garch_net.conditional_volatility) * gscale)
gtests
```

Output

```
omega      0.35
alpha[1]   0.18
beta[1]    0.45
Name: params, dtype: float64
```



Cell 64 · TEXT CELL

Text: what the GARCH result means (normality still fails because of a few extreme weeks). Heading 11.2.

Cell 65 · CODE CELL

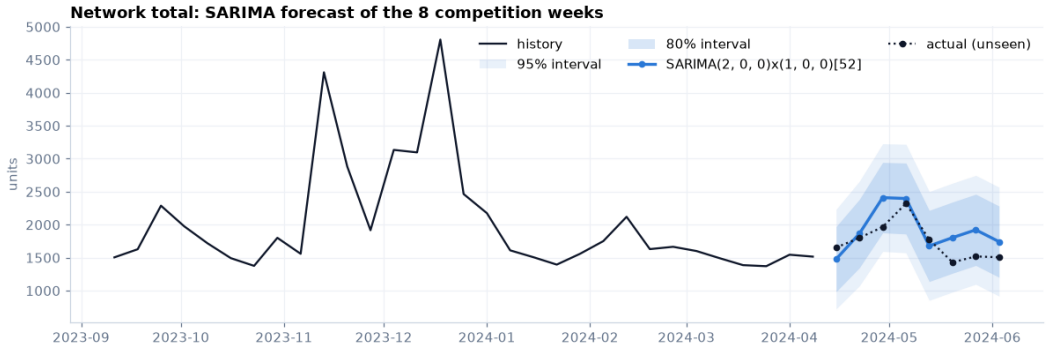
What it does. Uses the SARIMA fitted only on history to forecast the 8 competition weeks, with 80% and 95% ranges, and compares with what really sold and with "same week last year".
What you see. Error 14.2% vs 20.0%; all 8 real weeks fall inside the 80% range.
Why it matters. A real out-of-sample test: the model predicts weeks it never saw.

Code (start)

```
fc_net = fit_net.get_forecast(8)                          # fitted on history only, forecasts weeks 1-8
mean_net = np.asarray(fc_net.predicted_mean)
ci80, ci95 = np.asarray(fc_net.conf_int(alpha=.2)), np.asarray(fc_net.conf_int(alpha=.05))
actual_net = panel.sales.iloc[:, t0 + 1:t0 + 9].sum(axis=0).to_numpy()
last_year = total.iloc[t0 + 1 - 52:t0 + 9 - 52].to_numpy()                # "same week last year"
fc_dates = panel.dates[t0 + 1:t0 + 9]
plots.holdout_forecast(total.iloc[t0 - 30:t0 + 1], fc_dates, mean_net, ci80, ci95, actual_net, best_net["model"
])
… (5 more lines)
```

Output

```
Error (MAPE): SARIMA 14.2% · same week last year 20.0%
Real weeks inside the 80% range: 100%
```



Cell 66 · TEXT CELL

Heading 11.3: the same tests on every store-product.

Cell 67 · CODE CELL

What it does. Summarises the residual tests of the 599 ARIMA models, draws p-value histograms, then fits GARCH(1,1) to every product with an ARCH effect and re-tests.
What you see. 91% white noise; 10% ARCH effect; GARCH fixes 85% of those.
Why it matters. Shows the classical models are well specified across the whole chain.

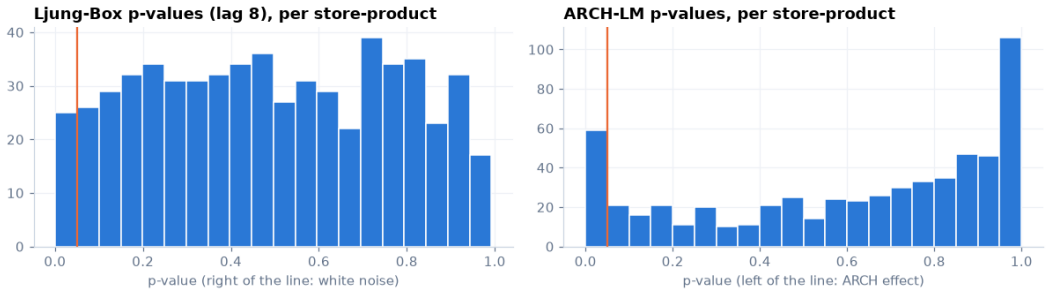
Code (start)

```
fitted = arima_tab.dropna(subset=["aic"])
wn = fitted["white noise"].astype(bool).mean()             # share passing Ljung-Box
arch_share = fitted["ARCH effect"].astype(bool).mean()     # share with volatility clustering
print(f"White-noise residuals: {wn:.0%} · ARCH effect: {arch_share:.0%}")
plots.pvalue_histograms(fitted["Ljung-Box p (lag 8)"], fitted["ARCH-LM p"])

season_h = M.season_of(M.season_factors(hist.upto(t0)), dates_h)
… (3 more lines)
```

Output

```
White-noise residuals: 91% · ARCH effect: 10%
GARCH(1,1) on the 59 store-products with an ARCH effect: clustering removed in 85% of them
```



Cell 68 · TEXT CELL

Text: summary of steps 4 to 11.

12 · Forecasting models

Cell 69 · TEXT CELL

Heading: step 12, forecasting methods. Why three weeks ahead, and a table of all 10 methods with their idea.

Cell 70 · TEXT CELL

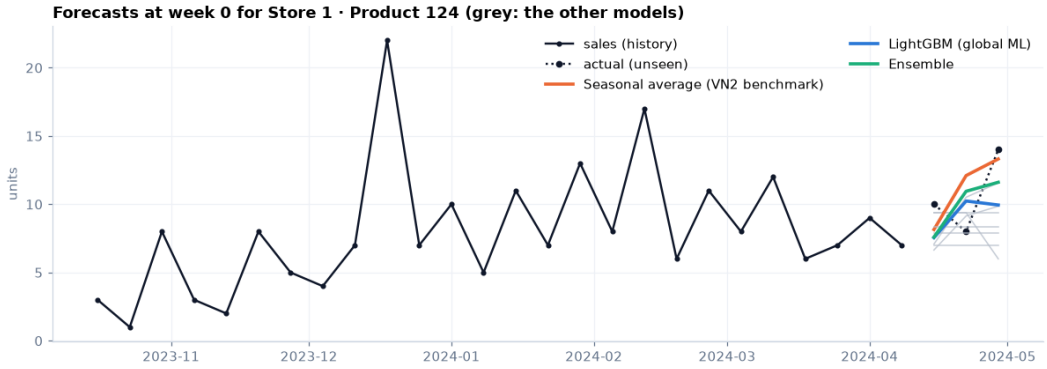
Text: LightGBM's input features and its Tweedie loss, and why.

Cell 71 · CODE CELL

What it does. Trains LightGBM once on all history, makes every method forecast the next 3 weeks for one example product, and plots them against the history and what really sold.
What you see. A chart with three highlighted methods and the others in grey.
Why it matters. Lets you see the methods side by side before measuring them properly.

Code (start)

```
lgbm_demo = M.GlobalLGBM(3, seed=SEED).fit(hist.upto(t0))  # one model trained on all store-products
demo = M.forecast_all(hist.upto(t0), 3, cfg.forecast.models, lgbm_demo, cfg.forecast.ensemble_of, arima_rows)
row = panel.index.get_loc(example)                         # the store-product shown in step 3
plots.model_forecasts(panel.sales.iloc[row, t0 - 25:t0 + 1], panel.sales.iloc[row, t0 + 1:t0 + 4],
                      {m: f[row] for m, f in demo.items()}, M.MODEL_LABELS,
                      f"Forecasts at week 0 for Store {example[0]} · Product {example[1]}")
```



13 · Backtest: which model forecasts best?

Cell 72 · TEXT CELL

Heading: step 13, the backtest and the four error measures (MAE, Bias, WAPE, Score).

Cell 73 · CODE CELL

What it does. The slowest cell (about 6 minutes). At each of 30 past dates, every method forecasts 3 weeks using only earlier data (LightGBM refitted every 4 dates, ARIMA every 8). All forecasts are compared with what sold.
What you see. The leaderboard table: MAE, RMSE, Bias, WAPE and Score for every method.
Why it matters. A fair test: no method ever sees the weeks it predicts.

Code (start)

```
# At each of the 30 dates, every model forecasts 3 weeks using only earlier data (about 6 minutes)
bt, bt_info = E.backtest(hist, origins, cfg.forecast.models, cfg.forecast.horizon, cfg.forecast.ensemble_of,
                         cfg.forecast.lgbm_retrain_every, SEED, arima_refit_every=cfg.forecast.arima_refit_ever
y)
lb = E.leaderboard(bt)                                     # MAE, RMSE, Bias, WAPE and Score per model
lb.insert(1, "label", lb["model"].map(M.MODEL_LABELS))
lb
```

Output: 1 table(s), shown in the notebook.

Cell 74 · CODE CELL

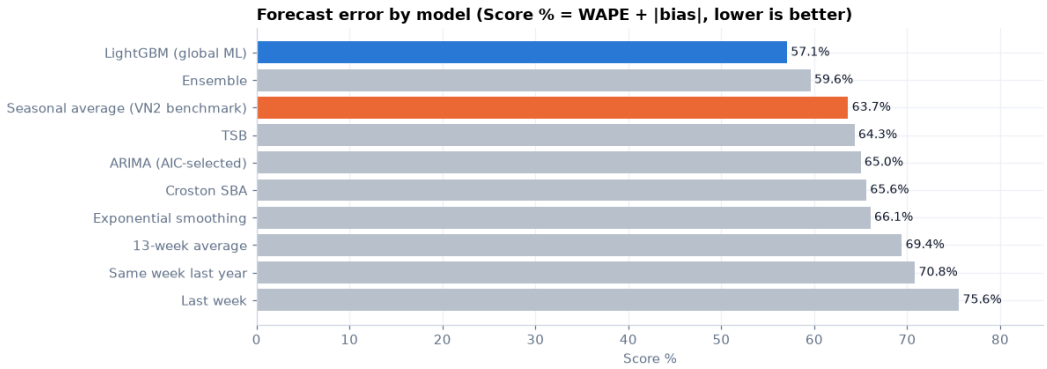
What it does. Draws the leaderboard as a bar chart.
What you see. LightGBM best (57.1%); the official benchmark forecast 63.7%; "last week" worst.
Why it matters. Shows clearly which forecast is most accurate.

Code (start)

```
plots.leaderboard(lb)
best_fc = lb.iloc[0]["model"]
bench_score = lb.set_index("model").loc["seasonal_ma", "Score %"]
print(f"Most accurate: {M.MODEL_LABELS[best_fc]} ({lb.iloc[0]['Score %']:.1f}%) · benchmark forecast {bench_sco
re:.1f}…
```

Output

```
Most accurate: LightGBM (global ML) (57.1%) · benchmark forecast 63.7%
```



Cell 75 · TEXT CELL

Text: next, accuracy by forecast horizon and by demand pattern.

Cell 76 · CODE CELL

What it does. Two tables: error 1, 2 and 3 weeks ahead, and error per demand pattern.
What you see. Further ahead is a little harder; intermittent and lumpy products are much harder.
Why it matters. Shows where forecasting struggles.

Code (start)

```
lb_h = E.leaderboard(bt, ["h"])                          # by weeks ahead
display(lb_h.pivot(index="model", columns="h", values="Score %").round(1))
bt_pat = bt.merge(classes[["pattern"]].reset_index(drop=True).rename_axis("row").reset_index(), on="row")
lb_p = E.leaderboard(bt_pat, ["pattern"])                # by demand pattern
lb_p.pivot(index="model", columns="pattern", values="Score %").round(1)
```

Output: 2 table(s), shown in the notebook.

Cell 77 · TEXT CELL

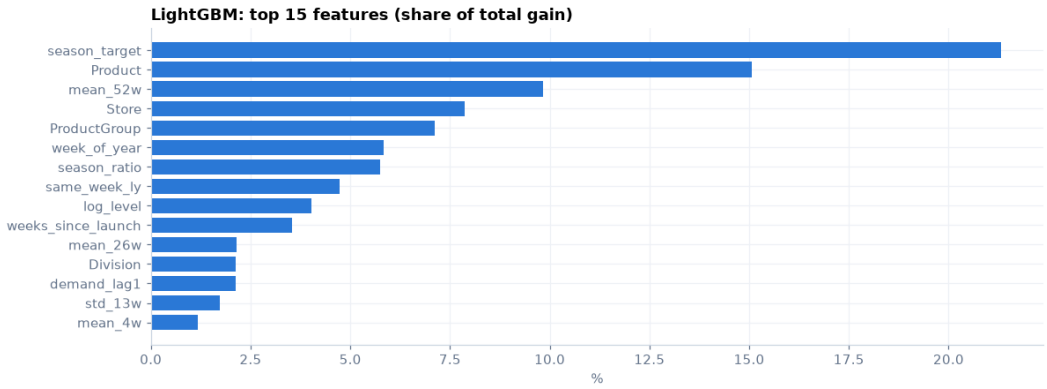
Text: reads the tables. Then: what drives LightGBM?

Cell 78 · CODE CELL

What it does. Charts LightGBM's 15 most important inputs (share of total gain).
What you see. Recent averages and lags lead; season and product groups follow.
Why it matters. Explains the machine-learning model in plain terms.

Code (start)

```
imp = bt_info["lgbm"].importance()                       # share of total gain per feature
plots.importance(imp)
```



Cell 79 · TEXT CELL

Text and heading 13.1: Ljung-Box on the real forecast errors of the backtest.

Cell 80 · CODE CELL

What it does. For every method and store-product, tests whether the 1-week-ahead errors over 30 weeks are white noise (Ljung-Box, lag 4).
What you see. Most methods: 89-93% of products pass; "last week": 52%.
Why it matters. A good forecast leaves no pattern in its mistakes. A pass does not prove accuracy; the leaderboard does that.

Code (start)

```
whiteness = TS.error_whiteness(bt, cfg.forecast.models, lag=4)   # Ljung-Box on each store-product's errors
whiteness.round(2)
```

Output: 1 table(s), shown in the notebook.

Cell 81 · TEXT CELL

Text: how to read the result, and why accuracy and white noise together are the evidence.

14 · Uncertainty: how wrong can a forecast be?

Cell 82 · TEXT CELL

Heading: step 14, uncertainty. Demand is modelled as a negative binomial with mean = forecast and variance k × mean.

Cell 83 · CODE CELL

What it does. Estimates the spread k for every product and method from backtest errors, then checks calibration: does demand stay below the 50%, 80% and 95% marks about 50%, 80% and 95% of the time?
What you see. A table: for example LightGBM 51%, 85%, 98%.
Why it matters. Without the spread, orders would cover only the average and run out about half the time.

Code (start)

```
disp_early = E.dispersion(bt[bt.origin.isin(disp_origins)], hist.n)   # spread k from dates 1-12 (for tuning)
disp_all = E.dispersion(bt, hist.n)                                   # spread k from all 30 dates (for the tes
t)
calib = E.calibration(bt[bt.origin.isin(tune_origins)], disp_early, M.MODEL_LABELS)
calib.round(2)
```

Output: 1 table(s), shown in the notebook.

15 · The inventory simulator (official rules)

Cell 84 · TEXT CELL

Text: the ranges are slightly generous, and step 16 corrects for that. Heading: step 15, the simulator, with the weekly rules written as five lines.

Cell 85 · TEXT CELL

Text: before trusting the simulator, reproduce the official benchmark score of €4,334.0.

Cell 86 · CODE CELL

What it does. Plays the organiser's published benchmark orders through our simulator, then rebuilds the benchmark method and compares its orders. Stops with an error if anything differs.
What you see. Setup €913.8 · score €4,334.0 · identical orders: True □ simulator verified, and the benchmark week by week.
Why it matters. Proves our scores are directly comparable with the real competition leaderboard.

Code (start)

```
costs = I.Costs(cfg.competition.holding_cost, cfg.competition.shortage_cost)   # €0.20 and €1
initial = I.State.from_initial(panel.initial)                                  # the real starting stock
comp_demand = panel.sales.iloc[:, t0 + 1:t0 + 9].to_numpy()                    # demand of weeks 1-8

# 1. The organiser's published benchmark orders
bench_orders = [o.to_numpy() for o in D.load_benchmark_orders(cfg.paths.raw_dir)]
bench_weekly, _, _ = I.simulate(initial, comp_demand, bench_orders, costs)
… (11 more lines)
```

Output

```
Setup €913.8 · score €4,334.0 · identical orders: True    simulator verified
```

16 · The ordering policy

Cell 87 · TEXT CELL

Text: the check passed. Heading: step 16, the ordering policy in 3 steps; its code is newsvendor_orders in src/optichain/inventory.py.

Cell 88 · TEXT CELL

Text: tuning by simulation, replaying 18 past weeks for every method and service level.

Cell 89 · CODE CELL

What it does. Replays the 18 past weeks for every method at 12 service levels, and the benchmark's order-up-to rule (function tune_service_levels), then keeps the cheapest setting per method (best_settings).
What you see. A table of the best service level and its replay cost per method (about 2.5 minutes).
Why it matters. Chooses the safety level from real simulated costs, not from a formula.

Code (start)

```
grid = cfg.policy.service_grid
tuning = P.tune_service_levels(bt, tune_origins, hist.sales, disp_early, costs, cfg.forecast.models, grid,
                               samples=cfg.policy.samples, seed=SEED, warmup=cfg.policy.replay_warmup)
best_service = P.best_settings(tuning, "newsvendor")       # cheapest service level per model
best_cov = P.best_settings(tuning, "order_up_to")          # cheapest coverage per model (benchmark-style rule)
best_service.sort_values("cost")
```

Output: 1 table(s), shown in the notebook.

Cell 90 · CODE CELL

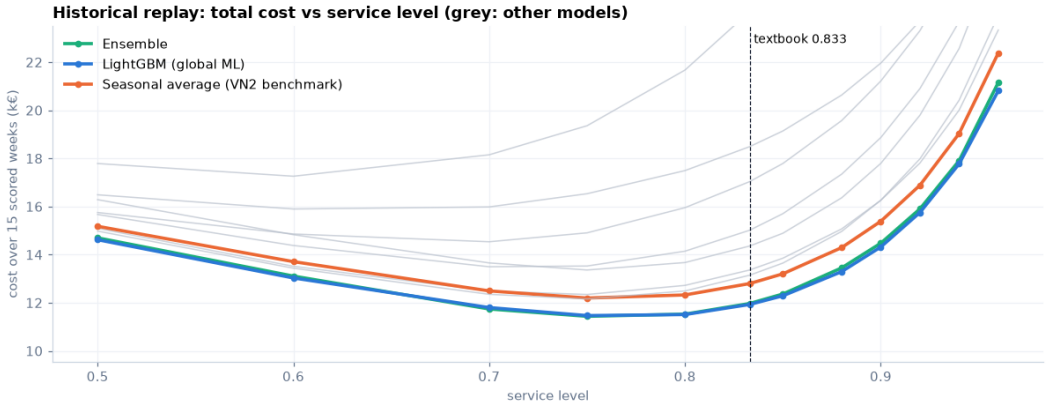
What it does. Draws cost against service level for every method and picks the overall winner.
What you see. U-shaped curves; the chosen plan is Ensemble at service level 0.75.
Why it matters. The bottom of the U is the best balance between empty shelves and leftover stock.

Code (start)

```
plots.tuning_curves(tuning[tuning.policy == "newsvendor"], M.MODEL_LABELS, costs.critical_ratio)
plan_model = best_service["cost"].idxmin()                # the model with the cheapest replay
plan_service = float(best_service.loc[plan_model, "param"])
print(f"Chosen plan: {M.MODEL_LABELS[plan_model]} at service level {plan_service}")
```

Output

```
Chosen plan: Ensemble at service level 0.75
```



17 · Final test: the eight VN2 weeks

Cell 91 · TEXT CELL

Text: better forecasts mean lower cost, and why 0.75 is below the textbook 0.833. Heading: step 17, the final test.

Cell 92 · CODE CELL

What it does. Makes forecasts for each of the 9 rounds (weekly, retraining LightGBM on the data known then) and measures accuracy on the competition weeks.
What you see. An accuracy table for the 8 test weeks (about 6 minutes to compute the forecasts).
Why it matters. Simulates exactly how the competition worked, week by week.

Code (start)

```
# Forecasts of every round, refitting on the data known then (about 6 minutes)
rounds = P.round_forecasts(hist, t0, cfg.forecast.models, cfg.forecast.ensemble_of, n_rounds=9, horizon=4, seed
=SEED)

# Accuracy on the competition weeks (1 to 3 weeks ahead, clean weeks only)
acc = rounds[(rounds.h <= 3) & (rounds.week <= 8)].copy()
acc["demand"] = demand.to_numpy()[acc["row"], t0 + acc["week"]]
lb_comp = E.leaderboard(acc)
… (2 more lines)
```

Output: 1 table(s), shown in the notebook.

Cell 93 · TEXT CELL

Text: an honest surprise. On these 8 weeks the simple benchmark forecast was the most accurate.

Cell 94 · CODE CELL

What it does. Plays the 8 competition weeks for every method with its service level tuned on the past, and also the order-up-to rule; records the score and the "hindsight best".
What you see. A table: Ensemble €3,626 is lowest; LightGBM €3,680; ARIMA €3,757.
Why it matters. The real test: what each forecast would have cost in the actual competition.

Code (start)

```
comp_scores, comp_grid = P.score_competition(initial, comp_demand, rounds, disp_all, costs, cfg.forecast.models
,
                                             M.MODEL_LABELS, best_service, best_cov, grid, cfg.policy.samples,
SEED)
comp_scores
```

Output: 1 table(s), shown in the notebook.

Cell 95 · CODE CELL

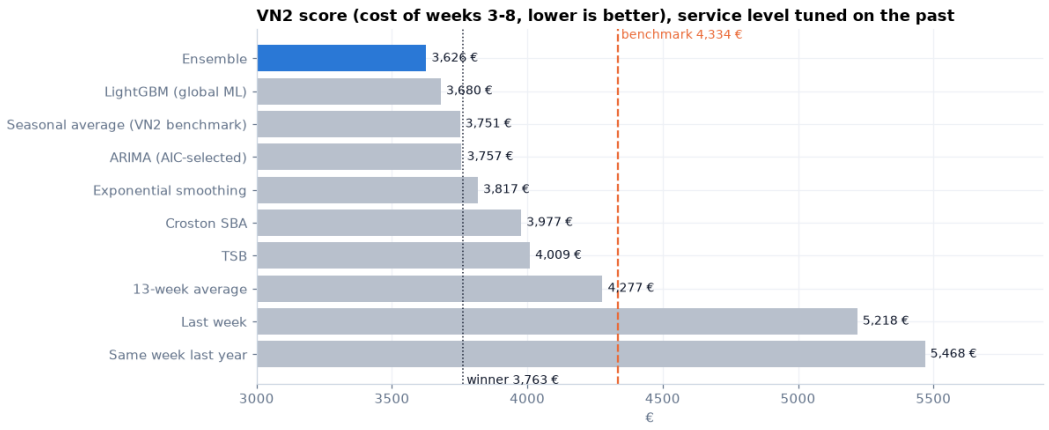
What it does. Charts every method's score against the official benchmark and the real winner.
What you see. Plan: Ensemble at 0.75, €3,626.4, 16.3% below the benchmark.
Why it matters. The headline result of the project.

Code (start)

```
plan_score = float(comp_scores.set_index("model").loc[plan_model, "VN2 score €"])
plots.competition_scores(comp_scores, plan_model, cfg.competition.benchmark_score, cfg.competition.winner_score
)
print(f"Plan: {M.MODEL_LABELS[plan_model]} @ {plan_service} → €{plan_score:,.1f} "
      f"({1 - plan_score / bench_official:.1%} below the benchmark)")
```

Output

```
Plan: Ensemble @ 0.75 → €3,626.4 (16.3% below the benchmark)
```



Cell 96 · TEXT CELL

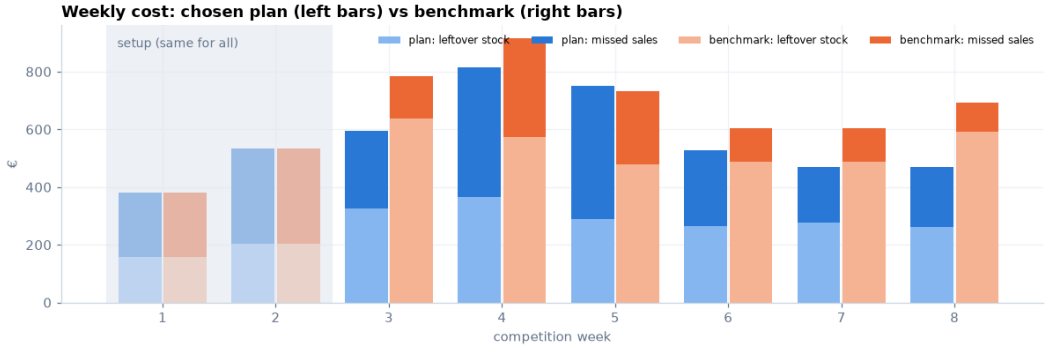
Text: how to read the result honestly (the hindsight caveat), and the weekly cost comparison.

Cell 97 · CODE CELL

What it does. Re-runs the chosen plan week by week (P.run_plan) and charts the weekly costs (leftover and missed sales) against the benchmark.
What you see. Stacked bars per week, and a table.
Why it matters. Shows where the saving comes from: much less leftover stock.

Code (start)

```
plan = P.run_plan(initial, comp_demand, rounds, P.k_vector(disp_all, plan_model), plan_model, plan_service, cos
ts,
                  cfg.policy.samples, SEED)
plots.weekly_costs(plan.weekly, bench_weekly)
pd.DataFrame({"plan cost": plan.weekly["cost"], "benchmark cost": bench_weekly["cost"]}).round(1)
```



18 · Stock outlook: the next order

Cell 98 · TEXT CELL

Heading: step 18, stock outlook. What to order after the data ends.

Cell 99 · CODE CELL

What it does. Builds a table for every store-product: stock on the shelf, stock on its way, forecast, order now, and the chance of running out in weeks 9, 10 and 11.
What you see. Order 1,162 units for 271 products; 19 products have a high risk next week.
Why it matters. Turns the analysis into a concrete action list for the planner.

Code (start)

```
out = P.outlook_table(panel.index, classes["pattern"].to_numpy(), plan, P.forecast_matrix(rounds, plan_model, 9
))
print(f"Order now: {out['order now'].sum():,.0f} units for {(out['order now'] > 0).sum()} store-products")
print(f"High risk (>50%) of an empty shelf next week: {(out['risk wk 9'] > .5).sum()} store-products")
out.sort_values("risk wk 9", ascending=False).head(10).round(2)
```

Output

```
Order now: 1,162 units for 271 store-products
High risk (>50%) of an empty shelf next week: 19 store-products
```

19 · Save the artefacts for the dashboard

Cell 100 · TEXT CELL

Text: weeks 9 and 10 are already decided; the new order protects week 11. Heading: step 19, save.

Cell 101 · CODE CELL

What it does. Saves every result (tables, forecasts, scores, time-series results) to the artifacts folder and lists the files.
What you see. "Saved N files" to the artifacts folder.
Why it matters. The dashboard reads these files, so the dashboard and the notebook always agree.

Code (start)

```
# Tables
files = {"audit.csv": audit, "leaderboard.csv": lb, "leaderboard_h.csv": lb_h, "leaderboard_pattern.csv": lb_p,
         "leaderboard_competition.csv": lb_comp, "calibration.csv": calib, "competition_scores.csv": comp_score
s,
         "benchmark_weekly.csv": bench_weekly, "outlook.csv": out, "error_whiteness.csv": whiteness,
         "tuning.csv": pd.concat([tuning, comp_grid]), "importance.csv": imp.rename("gain").rename_axis("featur
e"),
         "season.csv": season.rename("factor").rename_axis("iso_week")}
for name, table in files.items():
… (38 more lines)
```

Output

```
Saved 24 files to C:\Users\sanya\OptiChain\artifacts
```

20 · Conclusions

Cell 102 · TEXT CELL

Heading: step 20, conclusions.

Cell 103 · CODE CELL

What it does. Prints the conclusions with the real numbers filled in automatically.
What you see. Four lines: time-series result, forecasting, stock policy, final result.
Why it matters. Numbers come from the run itself, so the text can never be out of date.

Code (start)

```
print(f"Time series: the network total is stationary and seasonal (strength {s_net['seasonal strength']:.2f});
"
      f"best model {best_net['model']}; white noise after GARCH. Per store-product ARIMA: {wn:.0%} white noise.
")
print(f"Forecasting: {M.MODEL_LABELS[best_fc]} is the most accurate in the backtest "
      f"({lb.iloc[0]['Score %']:.1f}% vs {bench_score:.1f}% for the benchmark forecast).")
print(f"Stock policy: Monte Carlo newsvendor, service level {plan_service} tuned on past weeks.")
print(f"Result: {M.MODEL_LABELS[plan_model]} plan €{plan_score:,.1f} vs official benchmark €{bench_official:,.1
f} "
      f"({1 - plan_score / bench_official:.1%} lower); winner €{cfg.competition.winner_score:,.1f}.")
```

Output

```
Time series: the network total is stationary and seasonal (strength 0.93); best model SARIMA(2, 0, 0)x(1, 0, 0)[52
]; w…
Forecasting: LightGBM (global ML) is the most accurate in the backtest (57.1% vs 63.7% for the benchmark forecast)
.
Stock policy: Monte Carlo newsvendor, service level 0.75 tuned on past weeks.
Result: Ensemble plan €3,626.4 vs official benchmark €4,334.0 (16.3% lower); winner €3,763.2.
```

Cell 104 · TEXT CELL

Text: limitations and next steps, and the data citation.

The whole notebook in one table

Each step, the cells that do it, and its key result.

Step	Cells	Key result
0 · Setup	1-2	tools loaded, seed fixed
1 · Data collection	3-11	599 store-products × 165 weeks; files consistent; 43% zero weeks
2 · Exploratory data analysis	12-30	strong Christmas peak; about 1 unit a week typical; not normal; stores 0-59% out of stock
3 · Dealing with missing values	31-40	pre-launch and out-of-stock weeks set aside (100% of out-of-stock weeks = 0 sales)
4 · Stationarity: the ADF test	41-49	ADF: total stationary; 94% of products stationary
8 · Identify the model: AR, MA, ARMA, ARIMA or SARIMA?	50-68	the total looks like SARIMA
12 · Forecasting models	69-71	10 methods
13 · Backtest: which model forecasts best?	72-81	LightGBM most accurate (57.1% vs 63.7% benchmark)
14 · Uncertainty: how wrong can a forecast be?	82-97	negative binomial spread; ranges slightly generous
18 · Stock outlook: the next order	98-101	order 1,162 units for 271 products
20 · Conclusions	102-104	conclusions